

Define-XML Version 2.0 Completion Guidelines

Revision History

Version	Date	Summary
1.0 Draft	2018-10-02	Draft for PHUSE WG ODS leads and Steering Committee feedback
1.0 Draft	2018-10-26	Draft for PHUSE WG Steering Committee feedback
1.0 Draft	2018-11-28	Draft for PHUSE Public Review
1.0 Draft	2019-05-09	Draft for CG Team brief proof-read after addressing PR comments
1.0	2019-06-10	Final Delivery for PHUSE Release

Contents

1. Introduction	1	2.6.2.3. External Codelist Components	22
1.1. Purpose of this Document	1	2.6.3. General Considerations for Codelists	23
1.2. What Define-XML is, and What It's Not	1	Table 2.6.3.1: Situations where codelists are not expected	23
1.3. How to Use This Document	1	Table 2.6.3.2: Situations where codelists are expected	23
1.4. Glossary and References	2	Codelist Example 1: RACE codelist example to illustrate planned versus actual study data	24
2. Completion Guidelines Details	2	2.6.4. Variables without a pre-defined list of terms requiring a controlled terminology	24
2.1. Study-level	2	Codelist Guideline 1: Using Controlled Terminology in Codelist	24
2.1.1. Specifying Study-level Metadata	2	Codelist Example 2: Units collected for local laboratories tests	24
2.1.2. Components of Study-level Metadata	2	2.6.5. Enumerated and Decoded Codelists	25
2.1.2.1. Study-level Metadata	2	Codelist Guideline 2: Providing Decoded Codelist ..	25
2.1.3. General Considerations for Study-level Metadata ..	3	Codelist Example 3: Completion/Reason for Non-Completion Codelist	25
2.2. Supporting Documents-level	4	Codelist Example 4: Unit Codelist as Decoded Codelist	25
2.2.1. Specifying Supporting Documents-level Metadata ..	4	2.6.6. Related Codelists	25
2.2.2. Components of Supporting Documents-level Metadata	4	Codelist Guideline 3	25
2.2.2.1. Supporting Documents-level Metadata	4	Codelist Example 5: Parameter Categories	25
2.2.3. Annotated Case Report Form	5	2.6.7. Variable-level and Value-level Codelists	26
2.2.4. Supplemental Documents	5	Codelist Guideline 4: Codelist at Value-level	26
2.3. Dataset-level	5	Codelist Example 6: CGI Questionnaire	26
2.3.1. Specifying Dataset-level Metadata	5	Codelist Example 7: Analysis Adverse Event	27
2.3.2. Components of Dataset-level Metadata	6	Codelist Guideline 5: Codelist at Variable-level	28
2.3.2.1. Dataset-level Metadata / General Metadata ..	7	Codelist Example 8: WHO Motor Milestones	28
2.3.2.2. Dataset-level Metadata / Detailed Structure (Organization) Metadata	8	2.6.8. Codelist Versioning	28
Dataset-Level Example 1.a: Only standard variables used as a key variable	8	2.6.9. Sponsor-defined CT / Extension of CDISC CT ..	28
Dataset-Level Example 1.b: Non-standard / supplemental qualifier variable used as a key	9	Codelist Example 9: WHO Motor Milestones	28
2.4. Variable-Level Metadata	10	Codelist Example 10: Needle Insertion Sites	29
2.4.1. Specifying Variable-level Metadata	10	2.6.10. Codelist Subsets	29
2.4.2. Components of Variable-level Metadata	10	Codelist Guideline 6: Varying Codelist Subsets	29
2.4.2.1. Variable-level Metadata	10	Codelist Example 11: Unit Codelists	29
2.4.3. Origin Metadata	12	Codelist Example 12: Occurrence (--OCCUR) and subject-level population flag (--FL) variables	30
2.5. Value-level Metadata	14	Codelist Example 13: Epoch codelist	30
2.5.1. Specifying Value-level Metadata	14	2.6.10.1. Naming Codelist Subsets	31
2.5.2. Components of Value-level Metadata	14	Codelist Guideline 7: Codelist Subsets Naming Convention	31
2.5.2.1. Value-level Metadata / Special Considerations	14	Codelists Guideline 8: Single Codelist Subset	31
2.5.3. General Considerations for Value-level Metadata ..	16	Codelist Example 14: Usage of Codelist with the same list of terms	31
2.5.4. Relationship between Variable-level and Value-level metadata	16	2.6.11. External Codelists	32
2.5.4.1. DataType attribute	16	2.6.11.1. Elements of the External Codelists Metadata	32
2.5.4.2. Length attribute	16	Codelist Example 15: MedDRA coding for AE	33
2.5.4.3. Origin element	16	Codelist Example 16: Country Codes for DM domain COUNTRY variable	33
2.5.4.4. CodeList element	16	2.7. Methods	33
VLM Example 1: Example to illustrate VLM use cases for DataType, Length & Origin Type	17	2.7.1. Methods Metadata	33
2.5.5. Where Condition	17	2.7.2. Components of Methods Metadata	34
VLM Example 2: Example to illustrate Where Condition	18	2.8. Comments	34
VLM Example 3: VLM Use Case in TI domain For Different Protocol Versions	18	2.8.1. How Comments Work	34
VLM Guideline 1: Use of Coded Variable in Where Condition	18	2.8.2. Components of Comments Metadata	35
VLM Guideline 2: Grouping of VLM Using IN / NOTIN Operators	18	2.8.4. Examples of Comments Metadata	35
VLM Example 5: VLM Use Case for SF-36 V2.0 Standard Questionnaire	19	Appendix 1: Value-level Metadata Display Example Using Default Stylesheet	36
2.6. Codelists	20	Appendix 2: Completion Guidelines Development Team	37
2.6.1. Specifying Codelists	20		
2.6.2. Components of Codelists Metadata	21		
2.6.2.1. Codelist Metadata	21		
2.6.2.2. Codelist Terms Components	22		

1. Introduction ¹

1.1. Purpose of this Document

The intent of this document is to provide clarifications and examples on challenging metadata items to be represented in Define-XML version 2.0 documents, and how best to use the Define-XML 2.0 metadata items. Key concepts such as Codelist and Value Level Metadata (VLM) will include more examples and use cases. The analysis results metadata (extension of Define-XML 2.0 specification) is not covered in this document. Users are expected to be familiar with the CDISC Define-XML 2.0 Standard and other CDISC Data Standards (SDTM, SEND, ADaM and CDISC Controlled Terminology). The clarifications, guidelines, and examples provided in this document may not be fully appropriate for CDISC SEND IG in all instances. These guidelines only apply when creating Define-XML 2.0 metadata for electronic submissions. The differences between submissions to FDA and PMDA are not covered in this document. For guidance on actual regulatory submission of metadata and data, users are encouraged to refer to sources like the CDISC MSGs and any other applicable guidance documents from regulatory agencies (e.g., TCGs).

1.2. What Define-XML is, and What It's Not

Define-XML is a metadata standard mainly designed to describe the contents of SDTM, ADaM, SEND, or legacy tabulations or analysis datasets. The metadata in the Define-XML document is intended to describe the content of a database at the level of the study, datasets, variables, variable values, and codelists. It also contains comments about datasets, variables, or sets of variable values, as well as information about derivation methods and supporting documents like annotated CRFs and reviewer's guides.

A Define-XML document is a machine-readable, not a formatted, browse-able document. It's a stream of plain text only with embedded markup language, and contains no formatting for viewing or printing. Instead, formatting is handled by software viewers, like a stylesheet, that can interpret a Define-XML document that conforms to the Define-XML Specification. A stylesheet is a program that executes in a browser when users open a Define-XML document that contains a reference to that stylesheet, to make the document human-readable.

The Define-XML 2.0 standard does not include any presentation metadata (e.g. metadata to be used with a stylesheet for presentation purposes) and users can't control the viewable format of text included in a Define-XML 2.0 document. Therefore, if there is content that requires a different formatting than the one provided by the available default stylesheet, such as tables or complex equations, such content could be better placed in a separate supplemental document and reference it in the Define-XML document. See sections 2.7 Methods and 2.8 Comments for information about how to do that.

Note that, with the functionality of the default stylesheet, there is a practical size limit for displaying fields with the stylesheet. Fields that are too long will not be easily readable. Though there is no official rule limiting field lengths, Define-XML document creators should conduct a visual review of their content – as

displayed by the stylesheet – to ensure legibility.

1.3. How to Use This Document

The notes, examples and guidelines are intended to clarify some of the metadata items needed in regulatory submissions, not to rephrase the information contained in either the Define-XML 2.0 Specification or model implementation guides.

Under each section titled “Components of [xyz] Metadata”, in the corresponding table, where [xyz] refers to each high-level metadata component, only the metadata items for which clarifications, examples or guidelines where considered to be added in the current release of these guidelines are included:

- the column Metadata Item refers to the term as referenced in this Completion Guidelines document. For metadata items that have an equivalent Define-XML 2.0 Standard element or attribute, the same name and spelling has been used. In other cases, modified names have been used. For example, `DataType` is kept the same as in the standard but “Origin Type” is being used to reference the type of origin for a variable or value-level item.
 - Not all metadata items referencing an ID within this document have an attribute in the Define-XML 2.0 Specification document to represent it (e.g. `CodeListID`, `CommentID`). They are used in this document for illustration purposes only. Those should not be confused with the `OID` attribute of Define-XML Specification.
 - In cases where a reference to such metadata items is needed, the user will need to use the corresponding `OID` in the Define-XML document.
- the column Notes is intended to just add clarifications and references.
- the column Examples is intended to provide simple examples or a reference to examples provided in different sections of the document.
- the column Define-XML 2.0 Specification Reference includes the section(s) of the Define-XML 2.0 Specification where specific technical details are provided.
 - This column does not include a reference for metadata items that do not have a place in the Define-XML 2.0 Specification.

The document includes a series of guidelines identified as “[xyz] Guideline #: [abc]” under a few document sections, where [xyz] represents to the section containing the guideline and [abc] represents a short name for the guideline; e.g., “Codelists Guideline 1: Using Controlled Terminology in Codelist”. These guidelines are intended as best practices and each guideline includes, both a rationale for why it is intended as a best practice as well as one or more examples. Some of the examples presented in this document are displayed as it may be rendered by a stylesheet and are not necessarily representation of actual XML code.

1.4. Glossary and References

Term	Description
aCRF	Annotated CRF
CDISC CT	From the CDISC Glossary: CDISC Controlled Terminology is the set of CDISC-developed or CDISC-adopted standard expressions (values) used with data items within CDISC-defined datasets. CDISC, in collaboration with the National Cancer Institute's Enterprise Vocabulary Services (EVS), supports the controlled terminology needs of CDISC Foundational and Therapeutic Area Standards
CT	Controlled Terminology
C-code	NCI/EVS Code
Default stylesheet	The stylesheet developed by the PHUSE Define-XML 2.0 Stylesheet Recommendations project team, currently released for production. The screenshots displayed in this document are based on visualization using this default stylesheet.
Define-XML document (aka, define.xml)	A file created based on the Define-XML Standard
Define-XML Specification	The document that contains the specifications for the CDISC Define-XML Standard, as published by CDISC (https://www.cdisc.org/).
Define-XML Standard	CDISC Define-XML 2.0 Standard
EVS	Electronic Vocabulary System
Metadata Item	A conceptual term used to refer to a distinct piece of metadata regardless of where it is technically defined in the Define-XML Specification.
MSG	Metadata Submission Guidelines
NCI	National Cancer Institute
TCG	Technical Conformance Guide
Value-level definition	Metadata definition for a value-level item
Variable-level definition	Metadata definition for a variable-level item
VLM	Value Level Metadata
PHUSE Wiki Project page	https://www.phusewiki.org/wiki/index.php?title=Define-XML_V2.0_Completion_Guidelines_%26_Style_Sheet_Recommendations
PHUSE Define-XML 2.0 Stylesheet Recommendations	See project deliverable under https://www.phuse.eu/phuse-references , Optimizing the Use of Data Standards

2. Completion Guidelines Details

Guidelines are organized by sections named Study-level, Document-level, Dataset-level, Variable-level, Value-level, Codelists, Methods, and Comments.

2.1. Study-level

2.1.1. Specifying Study-level Metadata

Metadata items related to identification of the study and general characteristics of metadata version included in a given Define-XML document are described in the following sections.

2.1.2. Components of Study-level Metadata

2.1.2.1 Study-level Metadata

Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
Study ID	The ID of the Study metadata item should be universally unique, if at all possible, as opposed to all other IDs in the Define-XML document, which are unique only within the file. Note the Study ID is intended to universally identify the study and not set of datasets being described (e.g., the ID used should remain the same for any Define-XML document created for the study. Sponsors should find their own way to get a unique ID. The requirement for a universally unique ID comes from the ODM 1.3.2 specification 2, on which the Define-XML Specification is based. The Define-XML Standard does not specify how the ID value should be generated, and it can be derived in many ways based on business rules.	Sponsor Internet domain and study name separated with a slash ("phuse.eu/PHU001") Registry Identifier parameter value from the Trial Summary dataset (e.g., ClinicalTrials.gov Identifier of the study "ClinicalTrials.gov .NCT12345678") NASDAQ's #: ("nasdaq<#>/ PHU001")	5.3.4. Study Element, StudyOID attribute

Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
GlobalVariables	It is not a separate metadata item but a container for the items below		5.3.4.1. GlobalVariables Element
StudyName 3	The sponsor's internal name assigned to the study. In case there is no internal name, can be the same as ProtocolName.	PHUSE-NEU-001	5.3.4.2. StudyName Element
StudyDescription	The full study title from the protocol.	Phase II, randomized, multicenter, double-blind, placebo-controlled, two-arm parallel group trial of XYZ 1 mg QD for reducing the risk of headache in patients with chronic headaches.	5.3.4.3. StudyDescription Element
ProtocolName 3	The official identifier of the study as specified in the protocol.	PHU001	5.3.4.4. ProtocolName Element
MetaDataVersion	It is not a separate metadata item but a container for the items below. There could be multiple metadata version per study; however, there should be only one per Define-XML document.		5.3.5. MetadataVersion Element
Metadata Name	Name of the metadata described (e.g. "Study <StudyName> SDTM Data Definitions"). This attribute should uniquely identify each different MetadataVersion	Study PHU001, Data Definitions, Update 1	5.3.5. MetadataVersion Element, Name attribute
Metadata Description	Optional description of the metadata. See Section 2.1.3. General Considerations for Study-level Metadata (#1)	Data Definitions for PHU001 SDTM datasets. Updated version following FDA comments for the initial submission. Terminology Standard used: CDISC/NCI SDTM CT (2014-12-19)/ ADaM CT (2016-12-18). This metadata includes associated person AP* domains, which are created using SDTMIG-AP version 1.0.	5.3.5. MetadataVersion Element, Description attribute.
DefineVersion	Version of the Define-XML Standard used by the Define-XML document in question. Should be the official CDISC version. For Define-XML documents created under these guidelines, the value should be "2.0.0".	2.0.0	5.3.5 MetadataVersion Element, def:DefineVersion attribute.
StandardName	Name of the CDISC standard used for the study metadata. Allowed values (extensible): "SDTM-IG", "ADaM-IG", "SEND-IG"	SDTM-IG	5.3.5. MetadataVersion Element, def:StandardName attribute.
StandardVersion	Version of the CDISC data standard identified in StandardName. The version should be one of the versions supported by FDA as referenced in the FDA Data Standard Catalogue 4.	3.2	5.3.5 MetadataVersion Element, def:StandardVersion attribute.

2.1.3. General Considerations for Study-level Metadata

1. The Define-XML 2.0 Standard does not have structured metadata items to describe some information critical for understanding the submitted data package 5. Examples of such information are: version of controlled terminology used, plus any names of additional standards (e.g., SDTMIG-AP) used for specific domains/datasets 6, the submission type for which the Define-XML document is prepared, version of the file and a reason for an update if the file is resubmitted, or

state whether the metadata described in a file is a subset of the full study metadata (e.g. interim analysis data cut).

- The Metadata Description item, optional as per the Define-XML 2.0 Specification, can be used to include this critical information.
2. There should be only one of each StudyName, StudyDescription and ProtocolName metadata items per Define-XML document.

2.2. Supporting Documents-level

2.2.1. Specifying Supporting Documents-level Metadata

Users can refer to the revised general Section “4.5 Links to Supporting Documents” on the Define-XML 2.0 Specification for a general understanding of the concept of Supporting Documents-level Metadata.

A regulatory submission package can include several non-Define-XML documents. At the supporting documents-level, users can describe and reference those documents within a Define-XML document. Supporting documents that can be referenced from a Define-XML document include an aCRF, reviewer guides, etc. Users should follow the latest regulatory guidance to understand which supporting documents, if any, are required at the time of a submission.

2.2.2. Components of Supporting Documents-level Metadata

Document definitions are made up of the following metadata items:

Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
Document Reference	As a general rule, all supporting documents located in the same eCTD folder as the Define-XML document should be referenced from the Define-XML document. Datasets are not considered as supplemental documents. The Define-XML stylesheet uses these definitions to create bookmarks and hyperlinks to specific locations of the documents from Define-XML metadata items like comments, methods, and origins.		Definition in: 5.3.6.1 def:DocumentRef Element child 5.3.15 def:leaf Element child 5.3.6.1.1 def:PDFPageRef Reference in: 5.3.6 def:AnnotatedCRF Element 5.3.7 def:SupplementalDoc Element
AnnotatedCRF	This is a container to include the aCRF definition. See Section 2.2.3. Annotated Case Report Form		5.3.6 def:AnnotatedCRF Element
SupplementalDoc	This is a container to include the list of supplemental document(s) definition. See Section 2.2.4. Supplemental Documents		5.3.4.3 StudyDescription Element
Title	A description that is used as a label for any hyperlinks created with a define stylesheet.	“Annotated CRF” “Study Data Reviewer’s Guide”	5.3.15 def:leaf Element, child 5.3.15.1 def:title Element
href	A relative path using slashes as delimiters, plus the full filename. Because the external document is almost always in the same folder as the Define-XML document, it will usually be just the filename. The path, including the filename, is case sensitive.	“acrf.pdf” “csdrg.pdf”	5.3.15 def:leaf Element, xlink:href attribute
PDFPageRef	This metadata item allows users to create references to individual pages or named destinations. For information about page references for origin, see section 2.4.3 Origin Metadata. The default stylesheet will include a link that allows reviewers to quickly navigate to the desired section of the document.		5.3.6.1.1 def:PDFPageRef Element

Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
Ref Type	PhysicalRef or NamedDestination Named destinations are useful in documents that are frequently updated during the development of the Define-XML document, because they don't need to be updated or checked every time the document is changed.		5.3.6.1.1 def:PDFPageRef Element, Type attribute
PageRefs	Note that, in case of multiple pages, the separator is a space, not a comma.	For physical references: "1" "4 6 12" For named destinations: "ADTTE_Censor_Date_Imputation"	5.3.6.1.1 def:PDFPageRef Element, PageRefs attribute
FirstPage		For a range like "11-16": "11"	5.3.6.1.1 def:PDFPageRef Element, FirstPage attribute
LastPage		For a range like "11-16": "16"	5.3.6.1.1 def:PDFPageRef Element, LastPage attribute

2.2.3. Annotated Case Report Form

When using CRF origins and respective page references at the variable- or value-level, the user must provide an aCRF. Refer to the MSG for guidance on CRF annotation.

2.2.4. Supplemental Documents

If a supplemental document is provided, the Define-XML document should include a SupplementalDoc element with a reference to one or more documents.

It is very important to note that the inclusion of supplemental documents does not eliminate the requirement to submit a complete and informative Define-XML document corresponding to the submission datasets 7.

2.3. Dataset-level

2.3.1. Specifying Dataset-level Metadata

The Dataset Metadata in the Define-XML Specification is intended to describe a table structure (2-dimensional dataset), regardless of the content of the dataset.

During the planning stage of the study, sponsors are encouraged to include definitions for ALL planned datasets in the Define-XML document. However, there may be instances of a planned dataset ending up empty at the time of submission. As per the SDTMIG v3.2, empty datasets should not be included in

the submission and the Define-XML document for submission should only include definitions of datasets that are being submitted. The operational aspect of removing those empty datasets definitions from a Define-XML document is out of scope for this document. There may be additional guidance on this topic from the CDSIC MSG 2.0 team in the future.

Define-XML 2.0 documents can describe:

- a table corresponding to an SDTM-based Domain that contains standard variables, often referred to as a "parent domain".
- a supplemental qualifier dataset with each record attached to a "parent dataset", known as a SUPP-- dataset, that contains non-standard or supplemental qualifier variables for the domain of the "parent dataset".
- an ADaM dataset.
- a SEND dataset.
- datasets that may need to be split for size or other considerations.
- other tables, like Trial Design or RELREC datasets.

The SDTMIG v3.2, Section 3.2 "Using the CDISC Domain Models in Regulatory Submissions – Dataset Metadata", includes a description of the main metadata items to be provided for a given dataset. It refers to dataset names, descriptions, locations, structures, class, purpose, keys, and comments. See screenshot for Table 3.2.1 from the SDTMIG v3.2 below.

3.2.1. Table 3.2.1. SDTM Submission Dataset-Definition Metadata Example

Dataset	Description	Class	Structure	Purpose	Keys*	Locations
DM	Demographics	Special Purpose Domains	One record per subject	Tabulation	STUDYID, USUBJID	dm.xpt
MH	Medical History	Events	One record per medical history event per subject	Tabulation	STUDYID, USUBJID, MHDECOD	mh.xpt

Note that the dataset's keys are explicitly described in this document as part of the Variable Metadata and the description included in this section is only intended to describe the relationship of the dataset keys with the detailed structure of a dataset.

For purposes of the description on dataset's keys, the Medical History domain will be used as an example of metadata included in Table 3.2.1 and the Domain Definition Tables in the SDTMIG v3.2.

MH – Specification for Medical History Domain Model

mh.xpt, Medical History – Events, version 3.2. One record per medical history event per subject. Tabulation.

Variable Name	Variable Label	Type	Controlled Terms, Codelist or Format	Role	CDISC Notes	Core
STUDYID	Study Identifier	Char		Identifier	Unique identifier for a study.	Req
DOMAIN	Domain Abbreviation	Char	MH	Identifier	Two-character abbreviation for the domain.	Req
USUBJID	Unique Subject Identifier	Char		Identifier	Identifier used to uniquely identify a subject across all studies for all applications or submissions involving the product.	Req
MHSEQ	Sequence Number	Num		Identifier	Sequence number given to ensure uniqueness of subject records within a domain. May be any valid number.	Req
MHGRPID	Group ID	Char		Identifier	Used to tie together a block of related records in a single domain for a subject.	Perm
MHREFID	Reference ID	Char		Identifier	Internal or external medical history identifier.	Perm
MHSPID	Sponsor-Defined Identifier	Char		Identifier	Sponsor-defined reference number. Perhaps pre-printed on the CRF as an explicit line identifier or defined in the sponsor's operational database. Example: Line number on a Medical History page.	Perm
MHTERM	Reported Term for the Medical History	Char		Topic	Verbatim or preprinted CRF term for the medical condition or event.	Req
MHMODIFY	Modified Reported Term	Char		Synonym Qualifier	If MHTERM is modified to facilitate coding, then MHMODIFY will contain the modified text.	Perm

2.3.2. Components of Dataset-level Metadata

The Define-XML 2.0 Specification includes the metadata items listed on a) Table 2.3.2.1 Dataset-Definition / General Characteristics Metadata, to describe the general characteristics of a dataset; and b) Section 2.3.2.2 Dataset Definition / Detailed Structure (Organization) Metadata, to indicate the order of variables within the dataset as well as the dataset keys that correspond to the actual structure of the dataset.

The general properties of datasets are detailed in the Define-XML ItemGroupDef element, which contains the following metadata items:

2.3.2.1. Dataset-level Metadata / General Metadata

Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
Dataset Name	The name of the dataset.	"DM", "AE", "SUPPDM"	5.3.10. ItemGroupDef Element section, Name attribute
SAS Dataset Name	Applicable to datasets submitted using the SAS Transport V5 mechanism. See heading of this column. Corresponds to the name known to SAS and is the same as "Name".	"DM", "AE", "SUPPDM", "ADSL"	5.3.10. ItemGroupDef Element section, SASDatasetName attribute
Domain	Applicable to datasets that follow the CDISC SDTM or SEND standards. It should correspond to a value of the CDISC/CT DOMAIN codelist (C-code = C66734). For split datasets, refers to the Domain described per SDTMIG v3.2, Section 4.1.1.7 Splitting Domains. Domain is not applicable in the case of RELREC datasets. Until the topic is clarified in the SDTM model and corresponding implementation guides, Domain should not be provided for RELREC. Domain for SUPP-- datasets refers to the Domain of the parent dataset, which should correspond to the definition of the RDOMAIN variable for Supplemental Qualifiers datasets (SDTMIG v3.2, Section 8.4.1 Supplemental Qualifiers: SUPP-- Datasets)	Datasets with Name = "AE" and Name = "SUPPAE" have the same Domain value "AE". Datasets with Name = "FACM" and Name = "SUPPFACM" have the same Domain value "FA". Split datasets with Name = "QSCS" and Name = "QSMM" have the same Domain value "QS".	5.3.10. ItemGroupDef Element section, Domain attribute 4.1.1.3. Example of a Split SDTM dataset definition
Description	It corresponds to the Dataset Label	"Demographics", "Adverse Events"	5.3.10. ItemGroupDef Element section, child Description element, 5.3.10.1. Description Element
Structure	General description of the structure of records in a dataset, according to the columns defined as keys in the 2-dimensional table. See Detailed Structure / Organization term.	"One record per subject"	5.3.10. ItemGroupDef Element section, def:Structure attribute
Repeating	Indicates whether a domain may contain more than one record per subject (Repeating = "Yes") or only one record per subject (Repeating = "No").		5.3.10. ItemGroupDef Element section, def:Structure attribute
IsReferenceData	Flags a dataset as reference data (when the dataset does NOT describe subject data, IsReferenceData = "Yes") or subject data (when the dataset describes subject data, IsReferenceData = "No").	For Trial design datasets: "Yes"; For any subject-level data: "No"	5.3.10. ItemGroupDef Element section, IsReferenceData attribute
Class	Applicable to datasets that follow the CDISC SDTM, SEND and ADaM standards. Refer to NCI Codelist GNRLOBSC (C-code = C103329) for accepted values.	For Domain = "DM", "SPECIAL PURPOSE"; For Domain = "AE", "EVENTS"	5.3.10. ItemGroupDef Element section, def:Class attribute
Purpose	Only datasets that follow the CDISC SDTM, SEND and ADaM standards are expected in a submission. No other values are allowed.	SDTM and SEND: "Tabulation" ADaM: "Analysis"	5.3.10. ItemGroupDef Element section, Purpose attribute

Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
Comment	To be used wherever extra documentation can provide clarity on the meaning of the dataset. For example, the reason it is included or any special considerations that may require the reviewer's attention.	For split datasets corresponding to the QS domain: "QS is submitted as a split dataset. The split was done based on QSCAT as QSCG (CLINICAL GLOBAL IMPRESSIONS), QSCS (CORNELL SCALE FOR DEPRESSION IN DEMENTIA) and QSMM (MINI MENTAL STATE EXAMINATION)." A general comment about order of datasets included in the dataset package could be included; e.g. Datasets are ordered based on the sequence of dataset keys provided except where noted individually.	5.3.10. ItemGroupDef Element section, def:CommentOID attribute and 5.3.14. def:CommentDef Element
Location	Refers to the location and filename of the file, which should be submitted in the eCTD folder structure. Note the dataset file name is in lowercase.	ae.xpt	5.3.10. ItemGroupDef Element section, def:ArchiveLocationID attribute and 5.3.15. def:leaf Element

2.3.2.2. Dataset-level Metadata/ Detailed Structure (Organization) Metadata

The examples tables within this section lists the variables present on each dataset, the variable order number or sequence in the dataset, and the sequence of a key variable in the set of key variables, whether the variable is a standard variable or a non-standard or supplemental qualifier variable. The other characteristics will be described in more detail when describing the variables or value-level metadata.

Although details on the content of variable-level metadata is included on the Variable-level Metadata section in this document, the metadata displayed for the MH dataset below illustrates the Key Sequence used to completely describe the dataset structure.

A Stylesheet, or any other Viewer Software, that displays the contents of a Define-XML 2.0 document, should be capable of building a Table of Contents like Table 3.2.1 included in the SDTMIG v3.2. From the Table of Contents, links to the complete detailed dataset structure should be provided.

Dataset-Level Example 1.a: Only standard variables used as a key variable

Based on the metadata needed and the order of variables in the MH domain as defined by the SDTMIG v3.2, the following table presents an example of how the actual MH domain in a study is structured and described. Note that the Variable Order Number does not necessarily correspond to the order of keys used (Key Sequence [3] [4] [5]) to identify a unique record in the dataset. Note that the key variables in this example differ from the ones listed on the SDTMIG v3.2 (See screenshot on Section 2.3.1 on this document). The reason is that this example refers to the case where the sponsor does not code medical history and MHDECOD is not present in the dataset.

Dataset Name	Variable ID	Variable Order Number (Sequence of Variable in Dataset based on the SDTMIG)	Key Sequence
MH	STUDYID	1	1
MH	DOMAIN	2	2
MH	USUBJID	3	3
MH	MHSEQ	4	
MH	MHTERM	5	4
MH	MHCAT	6	
MH	PRESP	7	
MH	OCCUR	8	
MH	MHBODSYS	9	
MH	MHDTC	10	
MH	MHSTDTC	11	5
MH	MHENRF	12	

Note that the MH dataset structure described as “One record per medical history event per subject” for this particular example corresponds to the list of variables “STUDYID, USUBJID, MHTERM, MHSTDTC”. Below is a depiction of the way Structure and Keys are represented in the SDTMIG v3.2, Table of Contents for datasets.

Dataset	Description	Class	Structure	Purpose	Keys	Documentation	Location
MH	Medical History	EVENTS	One record per medical history event per subject	Tabulation	STUDYID, DOMAIN, USUBJID, MHTERM, MHSTDTC		mh.xpt

Dataset-Level Example 1.b: Non-standard / supplemental qualifier variable used as a key

This example illustrates the case for the MH dataset, where the sponsor is modelling two records for a medical history event to differentiate the date of onset and the date of diagnosis. The sponsor is including a variable “MHEVDTYPE” (Medical History Event Date Type) as a supplemental qualifier variable.

Dataset Name	Variable ID	Variable Order Number (Sequence of Variable in Dataset based on the SDTMIG)	Key Sequence
MH	STUDYID	1	1
MH	DOMAIN	2	2
MH	USUBJID	3	3
MH	MHSEQ	4	
MH	MHTERM	5	4
MH	MHCAT	6	
MH	PRESP	7	
MH	OCCUR	8	
MH	MHBODSYS	9	
MH	MHDTC	10	
MH	MHSTDTC	11	5
MH	MHENRF	12	
SUPPMH	QNAM = “MHEVDTYPE”		6

Note that the MH dataset structure described as “One record per medical history event per event date type per subject” for this particular example corresponds to the list of variables “STUDYID, USUBJID, MHTERM, MHSTDTC, QNAM.MHEVDTYPE”. In instances where supplemental qualifier variables are used as natural keys contributing to a domain, a stylesheet would display QNAM.<value of QNAM>. Below is a depiction of the way Structure and Keys using supplemental qualifier variables are represented in the SDTMIG v3.2, Table of Contents for datasets.

Dataset	Description	Class	Structure	Purpose	Keys	Documentation	Location
MH	Medical History	EVENTS	One record per medical history event per event data type per subject	Tabulation	STUDYID, DOMAIN, USUBJID, MHTERM, MHSTDTC, QNAM.MHEVDTYPE		mh.xpt

2.4. Variable-Level Metadata

2.4.1 Specifying Variable-level Metadata

Every variable in every dataset included in a submission should be declared in the define document. The metadata specified at the variable-level applies to all cells in a column within a table unless overridden by VLM.

2.4.2. Components of Variable-level Metadata

The general properties of variables are detailed in the Define-XML ItemDef and ItemRef elements, which contains the following metadata items:

2.4.2.1. Variable-level Metadata

Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
Variable Name	Indicates a dataset variable name.	USUBJID, AETERM	5.3.11 ItemDef Element, Name attribute. 5.3.8.1. ItemRef Element
DataType	Describes the variable's data type. There are several rules for populating DataType, including a list of allowable terms. For more detailed instructions on each term's use, see the Define-XML 2.0 Specification. The date and time data types represent the planned specificity of the collected data, and not an interpretation of the actual collected values.	Value = '2012-09-01T11:10' DataType = 'partialDateTime' Value = '2012-09' DataType = 'partialDate' Value = '2012---01T-:10' DataType = 'incompleteDatetime' Value of TEDUR = 'P90D' DataType = 'durationDateTime'	5.3.11 ItemDef Element, DataType attribute. 4.2.1 Data Type Considerations
Length	Describes the maximum expected variable length. Length should only be used when DataType equals "text", "integer", or "float". For DataType = "text", length equals the number of characters the variable can store. For DataType = "integer", length N means that the variable can contain integer numbers inside the interval $[-10^N(N), 10^N(N)]$. For DataType = "float", length N and significant digits S mean that the variable can contain numbers inside interval $[-10^N(N-S), 10^N(N-S)]$	DataType = 'integer': Maximum value is 5000 Length = "4" This variable can process and store numeric values less than 10000 (i.e. 10^4). DataType = 'float': Maximum value is 12786.5678, maximum precision is 0.0001. Length = "9" SignificantDigits = "4" This variable can process and store numeric values between -100000 (i.e. -10^5) and 100000 (i.e. 10^5) with four decimals. DataType = 'text' Longest text string to be represented = 'Drug A' Length = "6"	5.3.11 ItemDef Element, Length attribute. ODM v1.3.2 Specification, Section 3.1.1.3.6.
SignificantDigits	SignificantDigits describes the number of digits following the decimal point in a floating-point number, and defines the precision used when rounding. SignificantDigits must be populated when DataType = "float" but should not be provided for data types other than float.	Value = 10.2 SignificantDigits = "1", Value = 100.6943 SignificantDigits = '4' Value = 1.41421356237 SignificantDigits = '11'	5.3.11 ItemDef Element, SignificantDigit attribute
SASFieldName	Equals the corresponding SAS variable name. This item is required for submissions and must follow rules for variable names in SAS Transport files.	Variable Name = 'AESTDTC' SASFieldName = 'AESTDTC'	5.3.11 ItemDef Element, SASFieldName attribute.

Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
DisplayFormat	DisplayFormat supports data visualization of numeric float and date values. DisplayFormat for float is <Length>.<SignificantDigits>	Example 1: Value of 9.4 means Length = 9 and SignificantDigits = 4 Example 2: SAS date9 format means display a numeric ADaM date variable in the format DDMMYYYY).	5.3.11 ItemDef Element, def:DisplayFormat attribute.
Label	Describes the variable label.	Name = "STUDYID", Label = "Study Identifier"	5.3.10.1 Description Element, TranslatedText attribute.
Origin Type	Origins are made up of several different components in the XML. Origin structures are discussed in detail in the 2.4.3 Origin Metadata section of this document and the Define-XML 2.0 Specification.		5.3.11.3 def:Origin Element
CodeListID	A reference to the unique ID of a CodeList metadata item that defines Controlled Terminology for the variable. A codelist should be provided for any variable that has a permissible value set and must reference a codelist in the CodeList metadata. See the 2.6 Codelists section for details.	"ACN", "AGEGR1".	5.3.11.1 CodeListRef Element, CodeListOID attribute.
CommentID	A reference to the unique ID of a Comment metadata item. Note CommentID is not the same as the OID that is referenced in the Define-XML Specification. See Section 1.3 How to Use This Document. Comments are described in detail in the 2.8 Comments section.	"AGEU", "ARMCD".	
Order	Order provides the display order of the variables within the dataset. See Section 4.1.4 Order of Variables on the SDTMIG v3.2 for variable ordering requirements for in SDTM or Section 3 of the ADaMIG for information about variable ordering in ADaM datasets. If this attribute is not used, stylesheets will list variables in the order they appear in the XML.	Order = "1"	5.3.8.1 ItemRef Element, OrderNumber attribute.
KeySequence	KeySequence is populated for variables used as keys, in sequential order. KeySequence values must be sequential for the key to display correctly in the standard Define-XML 2.0 stylesheet. Note that there should not be duplicate records for a given set of key variable values. Refer to the SDTMIG v3.2, Section 3.2.1.1 Primary Keys. The user's selection of variables to be used as keys should align with the actual structure of the dataset.	If the key sequence is "STUDYID, USUBJID" then STUDYID gets a KeySequence of 1, and USUBJID gets a KeySequence of 2. Refer to Dataset-Level Example 1.b: Non-standard / supplemental qualifier variable used as a key.	5.3.8.1 ItemRef Element, KeySequence attribute.

Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
Mandatory	Mandatory describes whether variable values can be null or not. If Mandatory = "Yes", then null values are not allowed. Note that variables that are marked Required by the SDTMIG should have Mandatory = "Yes". Otherwise Mandatory = "No", unless the sponsor chooses to designate a variable as Required.	DM.SUBJID is a Required variable as per SDTMIG so Mandatory = "Yes". DM.AGE is an Expected variable, but the sponsor chooses to designate it as Required so, Mandatory = "Yes"	5.3.8.1 ItemRef Element, Mandatory attribute.
Role	Indicates how the variable is used within the dataset. Optional for SDTM standard domains, as the values provided by the SDTMIG or SENDIG are used. Required for SDTM custom domains. Not applicable for ADaM.		5.3.8.1 ItemRef Element, Role attribute.
Method ID	A reference to a unique Method ID defined in the Methods metadata. Note Method ID is not the same as the OID that is reference in the Define-XML Specification. See Section 1.3 How to Use This Document. A method is required whenever a variable's Origin = "Derived". For details about defining methods, consult the 2.7 Methods section.	"AE.AEENDY", "DM.AGE".	

2.4.3. Origin Metadata

As noted in the table above, Origins have complex structures that contain multiple metadata items and are governed by several rules. Each piece will be explored in detail, and examples will show how a full origin can be created from the various pieces.

There are two important considerations when deciding how to assign origins:

- When a variable has only one Origin Type for all its values, and the variable has VLM attached to it (e.g. all QVAL values are found on CRF pages), users have the following three options:
 - Provide Origin Type at variable-level only.
 - Provide Origin Type at value-level only.
 - Provide Origin Type at both variable-level and value-level.
- Multiple Origin Types for an individual item definition (variable-level or value-level) are not allowed as per Define-XML 2.0 Specification. If a variable has multiple Origin Types, then Origin Type should be left blank at the variable level and VLM should be used to describe multiple Origin Types.

The following table contains a complete list of allowable Origin Types and the metadata items that need to be populated for each. A reference to the component table that contains the metadata item is added to the Usage Notes.

Origin Type	Metadata Item	Usage Notes	Examples
Assigned	Origin Type	Origin Type = "Assigned" for variables that are filled by individual judgment (by an evaluator other than the subject or investigator), rather than collected as part of the CRF, eDT or derived based on an algorithm. This category includes variables that are populated from an external dictionary.	Variables related to medical dictionaries: CMDECOD, AEDECOD
		No other metadata items are required for Assigned origins. See section 2.4.2.1 Variable-level Metadata, Origin Type	Numeric counterparts of primary variable in character format: SEXN, RACEN, AVISITN Character counterparts of primary variable in numeric format: APERIODC BDS parameter related variables: PARAM, PARAMCD, PARAMN.
Derived	Origin Type	Origin Type = "Derived" for variables that are calculated by an algorithm or reproducible rule that is dependent upon other data values. Derived variables must reference a Method. See section 2.4.2.1 Variable-level Metadata, Origin Type	RFSTDTC: RFSTDTC = first date/time of study drug, for safety subject. CMENDY: End date minus DM.RFSTDTC. Add 1 if >= 0. AWTDIFF: Absolute difference between AWTARGET and ADY.
eDT	Origin Type	Origin Type = "eDT" for variables that are received via an electronic Data Transfer (eDT) of data streams such as laboratory, ECG, or IVRS. No other metadata items are required for eDT origins. See section 2.4.2.1 Variable-level Metadata, Origin Type	Results variables from a data stream: PCORRES, EGORRES. Variables qualifying data stream results: PCCAT, LBSTAT.
Protocol	Origin Type	Origin Type = "Protocol" for variables that are defined as part of trial design preparations. For example, ECG position of subject (EGPOS variable in EG) may be specified only in the protocol. No other metadata items are required for Protocol origins. See section 2.4.2.1 Variable-level Metadata, Origin Type	EGPOS
CRF	Origin Type	Origin Type = "CRF" for variables that appear directly on the Case Report Form included in the submission. CRF origins must contain references to locations within the CRF document. See section 2.4.2.1 Variable-level Metadata, Origin Type	Multiple page references: Origin Type = "CRF" Ref Type = "PhysicalRef" PageRefs = "20 21 22" Page range reference: Origin Type = "CRF" Ref Type = "PhysicalRef" FirstPage = "20" LastPage = "22"
	Ref Type	Ref Type = "PhysicalRef" when the origin's location is represented as page numbers. Ref Type = "NamedDestination" when the origin's location is represented as a named destination in the PDF. For more information on this distinction, see Section 2.2.2.1 Supporting Documents-level Metadata, Ref Type metadata item.	Named destination reference: Origin Type = "CRF" Ref Type = "NamedDestination" PageRefs = "VS"
	PDFPageRefs	When Ref Type = "PhysicalRef", PageRefs will be a space-delimited list of one or more page numbers. When Ref Type = "NamedDestination", PageRefs will be a named destination that has been declared in the CRF. This metadata item should not be used when referencing a page range. See section 2.2.2.1 Supporting Documents-level Metadata, PDFPageRef metadata item	
	FirstPage	When Ref Type = "PhysicalRef" and the user wants to reference a page range, FirstPage will equal the page number of the first page in the range. This metadata item should not be used when referencing individual page numbers. See section 2.2.2.1 Supporting Documents-level Metadata, FirstPage metadata item	

	LastPage	When Ref Type = “PhysicalRef” and the user wants to reference a page range, LastPage will equal the page number of the last page in the range. This metadata item should not be used when referencing individual page numbers. See section 2.2.2.1 Supporting Documents-level Metadata, LastPage metadata item	
Predecessor	Origin Type	Origin Type = “Predecessor” for a variable with data that is copied from a variable in another dataset. Description is required for Predecessor origins. See section 2.4.2.1 Variable-level Metadata, Origin Type	Variable is copied from another dataset: For ADSL.RACE, Origin Type = “Predecessor” Description = “DM. RACE”
	Description	Contains a reference to the variable that acts as the Predecessor to the variable described. This should be in the form <Dataset>.<Variable>	Variable is copied from another variable with a different name: For ADLB.AVISIT, Origin Type = “Predecessor” Description = “LB. VISIT” Variable is a copy of the contents of a specific source variable from a subset of source records: For ADSL.DCREAS, Origin Type = “Predecessor” Description = “DS. DSDECOD” Comment = “For records where DS.DSCAT = ‘DISPOSITION EVENT’ and DS.DSSCAT = ‘END OF STUDY’ and DS.DSDECOD is not equal to ‘COMPLETED’”

2.5. Value-level Metadata

2.5.1. Specifying Value-level Metadata

Users can refer to the revised general section “4.5 Value-Level Metadata Definitions” on the Define-XML 2.1 Specification for a general understanding of the concept of Value-Level Metadata. The revised general section is intended to be applicable regardless of the Define-XML Standard version.

2.5.2. Components of Value-level Metadata

As value-level and variable-level metadata have almost identical structure, the same general properties and their usage from the Variable-level Metadata section are applicable. Special considerations and examples that illustrate the relationship between the two concepts are provided in Table 5.2.1 “Value-level Metadata / Special Considerations” and the following sections.

Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
Variable Name	The variable name for which the value-level is to be described.	“QVAL”	5.3.11 ItemDef Element, Name attribute.
Value-level Name	A short name associated with the value-level. In the case of SUPP data, it would be the name of the non-standard variable.	“HOADMREL” Refer to all VLM examples	5.3.11 ItemDef Element, Name attribute.
SASFieldName	A SAS variable name, with the same requirements as the variable-level SASFieldName, usually corresponds to the value-level name. For example, in case of a SUPP dataset, this attribute contains the same value as the QNAM variable.	“HOADMREL” Refer to all VLM examples	5.3.11 ItemDef Element, SASFieldName attribute.

Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
Label	A short description of the value-level. In case of a SUPP dataset, this attribute contains the same value as the QLABEL variable. Optional for VLM but required in case of VLM for supplemental qualifiers.	“Hospitalization Related To” Refer to all VLM examples	5.3.10.1 Description Element, TranslatedText attribute.
Order	The order of the value-level could be used for programmatic generation of datasets when the value-level corresponds to the description of a non-standard/ supplemental qualifiers variable. Otherwise, the order specified could be used by viewer software to display the different value-level items.	Name = “STUDYID”, Label = “Study Identifier”	5.3.8.1 ItemRef Element, OrderNumber attribute
KeySequence	For VLM this attribute is used in situations when a variable stored in a SUPP dataset, is a key variable to a parent dataset.		5.3.8.1 ItemRef Element, KeySequence attribute.
Role	There is no need to provide Role for value-level items that do not represent a supplemental qualifier variable. For value-level items that represent a supplemental qualifier variable, Role should be set accordingly based on what is represented in QNAM. Therefore, in this instance, Role could be different to those specified at the variable-level.	For value-level items created for QVAL: Where QNAM=“LBCONVU” (Conventional Unit Used at Site) as it is a unit for a result: Role = “VARIABLE QUALIFIER” Where QNAM=“EXLOCOTH” (Other Location of Dose Administration), consistent with the Role for the EXLOC variable based on SDTM 1.4: Role = “RECORD QUALIFIER”	5.3.8.1 ItemRef Element, Role attribute.
Where Condition	Condition to be satisfied that clearly identifies the Variable values being described. Conditions can be simple (depending only on one dataset variable) or compound conditions (depending on multiple dataset variables). Compound conditions are formed by connecting simple conditions with the AND logical operator. A Where Condition in the Define-XML Standard is defined by using one RangeCheck element for simple conditions or multiple RangeCheck elements for compound conditions. Refer to Section 2.5.5 Where Condition below for details on the different Where Condition components and additional examples.	Simple conditions: • QNAM EQ “HOADMREL” • LBTEST IN (“TEST1”, “TEST2”) Compound conditions: • LBTEST EQ “[xyz]” AND LBNAM EQ “LOCAL” • LBSPEC EQ “BLOOD” AND LBTESTCD EQ “GLUC”	5.3.8.2 def:WhereClauseRef Element 5.3.9 def:WhereClauseDef Element 5.3.9.1 RangeCheck Element

2.5.3. General Considerations for Value-level Metadata

When specifying VLM attributes, the following considerations need to be taken into account:

1. All attributes which are defined as required in the Define-XML 2.0 Specification, Section 5, need to be provided both at the variable-level and value-level even if there are contradictions to what was specified in Section 4.4 Value Level Metadata Definitions 8. For example, attribute `DataType` must be always specified, and attribute `Length` must be specified unless item has a `DataType` where `Length` is not applicable.
2. Relationship between variable and value-level metadata. For details on this topic, refer to Section 2.5.4 Relationship between variable-level and value-level metadata.
3. Convenience in presentation of metadata to the reviewer.

2.5.4. Relationship between Variable-level and Value-level metadata

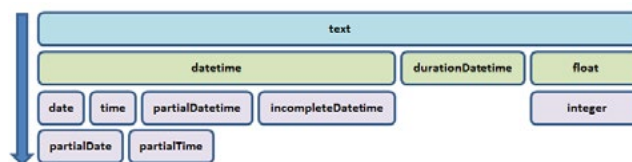
The Define-XML Specification states that it is not necessary to supply value-level metadata for properties that do not change at the value-level and which are not marked as required in the specification. Any optional metadata property 9, such as `codelist`, provided on a variable-level is applicable on a value-level unless overridden by a value-level property. For example, if at variable-level of an item definition, the user specifies 'NY' as a codelist and the codelist is not specified for an attached value-level definition, then this implies that the 'NY' codelist is also applicable at the value-level. As such, apart from the required attributes, it is not necessary to specify value-level properties which are the same as at the variable-level. However, if the repeating metadata can help to understand the variable it can be provided on the value-level.

Metadata specified at the value-level must not contradict the metadata specified at the variable-level it is attached to. Elements and attributes for which it is possible to specify metadata that does not match between variable-level and value-level, are listed and discussed below.

2.5.4.1. DataType attribute

It is common, that value-level `DataType` is different from the variable-level `DataType`. It does not violate the compatibility rule as some of the `DataType` can "include" other `DataType` values. For example, SDTM's `LBORRES` variable should have attribute `DataType` = "text". Still individual test values can have both numeric results such as 1 or 0.1 and character results such as 'NEGATIVE'. If value-level items are created, their `DataType` property should be set to "integer", "float" and "text" correspondingly so that these metadata characteristics could be represented accurately.

Compatibility of the `DataType` values going from variable-level to value-level is in a single way direction, represented by the diagram below. It means that if a variable-level `DataType` is "datetime" it can be "date" and "time" on the value-level, but not the other way around. If three value-level items are created for `LBORRES` based on the type of result, one for float results, one for integer results and one for character results, then the variable-level `DataType` must be "text", as it is the only `DataType` value which covers those three `DataType` values.



2.5.4.2. Length attribute

The compatibility concept mentioned above extends to the `Length` attribute. Value of `Length` at the value-level must not exceed the `Length` value at a variable-level it is attached to.

2.5.4.3. Origin element

When `Origin` is provided both on the variable-level and value-level, the `Origin Type` must either:

- Exactly match for non-CRF origins. For example, if at variable-level it is defined as "eDT", then all value-level `Origin Type` values should be "eDT".
- Variable-level should be inclusive of all value-level pages for CRF origins. For example, if at value-level three value-level items were created at CRF Pages 1, 3, and 9 then at variable-level all of these pages must be included, i.e. CRF Pages 1, 3, and 9.

If value-levels have different `Origin Type` values, then the `Origin Type` at the variable-level must not be provided. Multiple origins for a single variable-level or value-level item are not allowed as per Define-XML 2.0 Specification.

2.5.4.4. CodeList element

If a codelist needs to be provided on the value-level, see Section 2.6.7. Variable-level and Value-level Codelists in this document that contains details on how codelists should be defined in this case.

The value-level metadata examples presented in this document are not necessarily displayed the way certain style sheet would display it in the define. An example of how a stylesheet would display value-level metadata is presented in Appendix 1: Value-level Metadata Display Example Using Default Stylesheet.

VLM Example 1: Example to illustrate VLM use cases for DataType, Length & Origin Type**Variable-level Metadata:**

Variable Name	Label	DataType	Length/ DisplayFormat	Origin Type	Source/MethodDef/ CommentDef
AVAL	Analysis Value	float	Length = 4 DisplayFormat = 4.1		

Value-level Metadata:

Variable Name	Label	DataType	Length/ DisplayFormat	Origin Type	Source/MethodDef/ CommentDef
AVAL	PARAMCD EQ "HEIGHT"	float	Length=4 DisplayFormat=4.1	Predecessor	VS.VSORRES where VS.VSTESTCD EQ "HEIGHT"
AVAL	PARAMCD IN ("DIABP", "SYSBP")	integer	Length=3	Predecessor	VS.VSORRES where VS.VSTESTCD IN ("DIABP", "SYSBP")
AVAL	PARAMCD EQ "PULSEPR"	integer	Length=2	Derived	(AVAL where PARAMCD = "SYSBP") (AVAL where PARAMCD = "DIABP").

DataType & Length:

In the example above, DataType is specified at a value-level as this is a required attribute. Length is specified as well since when DataType is "float" or "integer". Length is also a required attribute.

Origin Type:

The Origin Type for AVAL with PARAMCD value of "PULSEPR" is 'Derived' while Origin Type for other records is "Predecessor". The multiple origins can't be presented at a variable-level in Define-XML 2.0 standard. Secondly, derivation algorithm is only applicable to PARAMCD value of "PULSEPR". Therefore, origin and derivations are not provided at a variable-level and present at value-level instead.

2.5.5. Where Condition

As VLM is variable metadata in relationship to other variable values, it is important to understand how the relationship is expressed. In the Define-XML Specification the relationship is described by the Where Condition metadata item. The Where Condition is a data slice constructed via the syntax as presented below:

<Variable> <Comparator> <Value> AND <Variable> <Comparator> <Value>

Optional: Multiple where conditions connected using the 'AND' operator as needed to describe the data slice

VLM Example 2: Example to illustrate Where Condition

Variable Name	Where Condition	Data Type	Length/DisplayFormat	CodeList
LBSTRESC	LBTESTCD EQ "GLUC"	float	Length=2 DisplayFormat=2.1	
LBSTRESC	LBMETHOD EQ "MICROSCOPY" AND LBTESTCD IN("BACT", "CSRAN")	text	Length=8	["mmol/L"] <Unit_Gluc>
LBSTRESU	LBTESTCD EQ "GLUC"	text	Length=6	

The Where Condition for variable LBSTRESU is exactly same as first Where Condition for variable LBSTRESC and it represents the same data slice. The VLM for LBSTRESU provides specific details about standard unit for the "GLUC" test code.

VLM Example 3: VLM Use Case in TI domain For Different Protocol Versions

Variable Name	Where Condition	Data Type	Length/DisplayFormat	CodeList	Origin Type
IETESTCD	TIVERS IN ("ORIGINAL", "AMENDMENT 1")	text	Length=8	IETESTCD_ORA1 ["EXC01_OR", "INC01_OR", "INC02_OR"]	Protocol
IETESTCD	TIVERS EQ "AMENDMENT 1.3"	text	Length=8	IETESTCD_A1_3 ["EXC01_OR", "INC01_OR", "INC02_13"]	Protocol

In the example above, second protocol inclusion criteria "INC02_OR" is updated in protocol amendment 1.3. As a result, new IETESTCD value of "INC02_13" is assigned. To display varying codelist values of IETESTCD across different versions of protocol, VLM has been used with Where Condition based on protocol version variable (TIVERS).

VLM Guideline 1: Use of Coded Variable in Where Condition

If there is a choice between coded and decoded variable for Where Condition, use coded variable.

Rationale: Keeps the Where Conditions concise. A stylesheet may add decodes, when a corresponding coded variable has a decoded codelist associated with it. For example, in VLM example 2 above, LBTESTCD variable is used in the Where Condition. However, the stylesheet also displays "(Glucose)" using decoded values from LBTESTCD codelist.

VLM Guideline 2: Grouping of VLM Using IN / NOTIN Operators

For a given variable, group value-level items with the same metadata using IN and NOTIN operator.

Rationale: Avoid metadata repetition. In VLM example 1, the metadata for data slice with PARAMCD values of DIABP and SYSBP is the same. Therefore, Where Condition is written as "PARAMCD IN ("DIABP", "SYSBP")" instead of writing two separate Where Conditions for each PARAMCD.

VLM Example 5: VLM Use Case for SF-36 V2.0 Standard Questionnaire

The last two columns in the table below shows allowed values for QSORRES and QSSTRESC/QSSTRESN based on the SDTM IG V3.2 guidance notes.

QSTESTCD	QSTEST	QSCAT	QSSCAT	QSORRES	QSSTRESC/N
S360201	S3602-Would You Say Your Health Is	SF36 V2.0 STANDARD	GENERAL HEALTH	EXCELLENT	1
					2
				VERY GOOD	3
				GOOD	4
				FAIR	5
S360202	S3602-Health Now Compared to 1 Year Ago	SF36 V2.0 STANDARD	HEALTH TRANSITION	POOR	
				MUCH BETTER NOW THAN ONE YEAR AGO	1
					2
				SOMEWHAT BETTER NOW THAN ONE YEAR AGO	3
					4
S360211A	S3602-Get Sick Easier Than Other People	SF36 V2.0 STANDARD	GENERAL HEALTH	ABOUT THE SAME AS ONE YEAR AGO	5
				SOMEWHAT WORSE NOW THAN ONE YEAR AGO	
				MUCH WORSE NOW THAN ONE YEAR AGO	
S360211B	S3602-I Am as Healthy as Anybody I Know	SF36 V2.0 STANDARD	GENERAL HEALTH	DEFINITELY TRUE	1
					2
				MOSTLY TRUE	3
					4
S360211C	S3602-I Expect My Health to Get Worse	SF36 V2.0 STANDARD	GENERAL HEALTH	DON'T KNOW	
				MOSTLY FALSE	
S360211D	S3602-My Health Is Excellent	SF36 V2.0 STANDARD	GENERAL HEALTH	DEFINITELY FALSE	5

The variable-level metadata for above example would not be sufficient to show metadata differences for variables QSORRES & QSSTRESC. The VLM for these variables can be presented to highlight those metadata differences.

The following VLM presentation allows reviewers to understand various response codelists based on different QSTESTCD values. The example below does not include all values of QSTESTCD.

Variable Name	Where Condition	Data Type	Length/ Display Format	Controlled Terms or Format	Origin	Derivation/ Comment
QSORRES	QSSCAT EQ GENERAL HEALTH AND QSTESTCD EQ S360201 (S3602-Would You Say Your Health Is)	text	9	QS_SF36_ABILITY_C	CRF Page 11	
QSORRES	QSSCAT EQ GENERAL HEALTH AND QSTESTCD IN ("S360211A" (S3602-Get Sick Easier Than Other People), "S360211B" (S3602-I Am as Healthy as Anybody I Know), "S360211C" (S3602-I Expect My Health to Get Worse), "S360211D" (S3602-My Health Is Excellent))	text	16	QS_SF36_TRUE_FALSE_C	CRF Page 11	

The following VLM presentation allows reviewer to understand original numeric responses based on different QSTESTCD values.

Variable Name	Where Condition	Data Type	Length/ Display Format	Controlled Terms or Format	Origin	Derivation/ Comment
QSSTRESC	QSSCAT EQ GENERAL HEALTH AND QSTESTCD EQ S360201 (S3602-Would You Say Your Health Is)	integer	1	QS_SF36_ABILITY_N	Derived	QSORRES in standard coded format
QSSTRESC	QSSCAT EQ GENERAL HEALTH AND QSTESTCD IN ("S360211A" (S3602-Get Sick Easier Than Other People), "S360211B" (S3602-I Am as Healthy as Anybody I Know), "S360211C" (S3602-I Expect My Health to Get Worse), "S360211D" (S3602-My Health Is Excellent))	integer	1	QS_SF36_TRUE_FALSE_N	Derived	QSORRES in standard coded format

2.6. Codelists

2.6.1. Specifying Codelists

In the context of this document, the term “Controlled Terminology” (CT) refers to a list of predefined standard terms maintained and published by a standards organization such as CDISC for standards such as CDASH, SDTM, SEND, and ADaM, or other terms defined and supported by the sponsor. As specified in the CDISC Implementation Guides, some variables by default have CDISC-published Controlled Terminology associated with them. Other variables may be associated with sponsor-defined Controlled Terminology. In the context of a study, it is unlikely that all of the CDISC Controlled Terminology terms are used on a specific variable or value-level item, which is why a study-specific Controlled Terminology is described within the Define-XML document.

A codelist in the context of a Define-XML document is often a study-specific subset of Controlled Terminology to which one or more variables are subject to and may consist of the following components:

- published controlled terminology applicable to the study, subsets or full sets of individual codelists,
 - with potential sponsor-defined extensions where allowed
- entirely sponsor-defined controlled terminology

2.6.2. Components of Codelists Metadata

The tables below describe all components for codelists.

Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
CodeListID	Short name for the codelist. As used in this document, for CDISC CT, it refers to the CDISC Submission Value for a codelist that can be found in the published CDISC CT. The Define-XML Specification does not provide an attribute to include this Metadata Item. Note CodeListID is not the same as the OID that is referenced in the Define-XML Specification. See Section 1.3 How to Use this Document.	See codelist example “Codelist Example 11: Unit Codelists”	
CodeList Name	Long name for the codelist. As used in this document, for CDISC CT, it refers to the Codelist Name that can be found in the published CDISC CT. This attribute is required. In case a codelist from the CDISC CT is used, the name of the codelist should exactly match the name from the CDISC CT. In other situations, user should avoid naming their codelists with the names from the CDISC CT. In case there are two or more codelists in the study which are different subsets of the same CDISC codelist, refer to Section 2.6.10.1 Naming Codelists Subsets.	See codelist example “Codelist Example 11: Unit Codelists”	5.3.12. CodeList Element, “Name” attribute.
DataType	This attribute is required. Describes the data types of the terms and corresponds to the type of a variable to which the codelist is applied. As variables with datetime values do not have a codelist associated with them, value of this attribute must be one of the following Define-XML data type values: text, float, integer.		5.3.12. CodeList Element, “DataType” attribute.
SASFormatName	This attribute is optional. A SAS format name which can be created from the codelist. Should be used in situations when decodes are provided. In case codelist DataType is text, it should begin with \$. If the attribute is specified, it does imply that the variable to which the codelist is assigned should have this format set for it.		5.3.12. CodeList Element, “SASFormatName” attribute.
Enumerated Codelist	See Section 2.6.5 Enumerated and Decoded Codelists.		5.3.12.1. EnumeratedItem Element
Decoded Codelist	See Section 2.6.5 Enumerated and Decoded Codelists.		5.3.12.2. CodeListItem Element
External Codelist	See Section 2.6.11 External Codelists.		5.3.12.1. EnumeratedItem Element, def:ExtendedValue attribute. 5.3.12.2. CodeListItem Element, def:ExtendedValue attribute.

2.6.2.2. Codelist Terms Components

Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
CodedValue	Short name for the codelist term. As used in this document, for CDISC CT, it refers to the CDISC Submission Value for a codelist term that can be found in the published CDISC CT. This attribute is required. This attribute contains the term value. As already mentioned it must exactly match the value of a variable to which the codelist is assigned, including case and must not be blank. See Section 2.6.5 Enumerated and Decoded Codelists.		5.3.12.2. CodeListItem Element
Rank	This attribute is optional. Numeric significance of the term relative to other terms from the same codelist. If the attribute is provided for one of the terms in a codelist it must be provided for all of the terms in the codelist. This attribute should be used when all of the terms can be compared between each other and it is not clear from the term values how to compare them. See Section 2.6.5 Enumerated and Decoded Codelists.		5.3.12.2. CodeListItem Element 4.3.1.4. Example of a CodeList using Rank attribute.
Decoded Value	This reflects association with CodedValue and necessary to have a 1-to-1 relationship		5.3.12.3. Decode Element 4.3.1.2. Example of CodeLists for VSTEST and VSTESTCD
OrderNumber	This attribute is optional. This attribute allows to specify an order which is used when the terms are shown to a user. If the attribute is provided for one of the terms in a codelist it must be provided for all the terms in the codelist. Comparing to the Rank attribute, values of this attribute do not necessarily imply anything about how the terms are compared between each other and are used only for the representation purposes.		5.3.12.2. CodeListItem Element, OrderNumber attribute.
ExtendedValue	A conditionally required attribute. In case the term is used to extend a standard CDISC CT value of the attribute must be set to "Yes". In all other cases this attribute should not be used. See the Sponsor-defined CT/Extension of CDISC CT section for details.		5.3.12.2. CodeListItem Element

2.6.2.3. External Codelist Components

Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
Dictionary Name	See Section 2.6.11.1 Elements of the External Codelists Metadata.		5.3.12.4. ExternalCodeList Element
Dictionary Version	See Section 2.6.11.1 Elements of the External Codelists Metadata.		5.3.12.4. ExternalCodeList Element 4.3.1.5. Example of a Codelist with ExternalCodeList child element
External Reference	See Section 2.6.11.1 Elements of the External Codelists Metadata.		5.3.12.4. ExternalCodeList Element

2.6.3. General Considerations for Codelists

1. In addition to variables subject to controlled terminology as per CDISC IGs and sponsor-specific controlled terminology, codelist should also be provided for all other variables and value-level definitions which have a predefined and finite set of categorical allowable values.
 - a. Refer to Section 2.6.4 Variables without a predefined list of terms requiring a controlled terminology for other cases when controlled terminology is applicable.
2. All controlled terms from the codelists, as originally planned for the study, should be included, whether they are represented in the submitted data or not. In addition, new controlled terms added during trial conduct should be included. See Codelist Example 1: RACE codelist example to illustrate planned versus actual study data and Codelist Example 2: Units collected for local laboratories tests.
3. In some situations, it may be challenging to understand what a predefined and categorical finite set is.
 - a. To understand whether a codelist should be applicable to a numeric variable, in most situations it is enough to check if a meaningful decode can be provided for each of the numeric values.
 - b. Note that the number of values in a codelist must not be used as a criterion when deciding whether to provide a codelist or not.
 - i. It is a common situation, when a codelist consists only of one value, as well as it is possible to have hundreds of values defined in a codelist.
 - c. Reviewers need to understand when a variable coming from a CRF has a limited number of choices or a variable is derived into a set of values each of which has a specific meaning.
4. Values in the data must exactly match values in the codelist if one is provided, including case.
5. Missing or blank values are not valid and must not be part of a codelist. If it is required to specify that a variable or value-level does not allow missing values, a corresponding "Mandatory" attribute in a variable or value-level definition should be used.
6. The ISO 8601 format is not a codelist nor an external codelist. This refers to the format used for date, time and duration related DataTypes and is displayed with the default Define-XML v2.0 Stylesheet.
7. The ISO 3166 list of country codes is considered an external codelist. See Codelist Example 16: Country Codes for DM domain COUNTRY variable.

The tables below provide examples of when a codelist is typically not expected and when it is typically expected. In any case, each instance needs to be evaluated by users due to potential exceptions.

Table 2.6.3.1: Situations where codelists are not expected

Situation	Rationale
Age variable for a study where participants are 18-75 years old	Although values of the age variable are integers and the age variable has only a finite set of values from 18-75, addition of the codelist does not bring any value for the reviewers.
Total distance measured for the 6-minute Walk Functional Test	Depending on distance achieved during each minute within the 6-minute walk, the derived total distance would be different therefore codelist is not expected.
QS Summary Score which represents a calculation of individual collected scores	Each individual score has a finite number of values which means that the summary score also has a finite number of values. Providing a codelist for it does not add any clarification and should be avoided.
Numeric measurements for vital signs	Continuous or quantitative results are not a finite set therefore codelist should not be created.
Duration of laboratory measurements	In SDTM, durations, as well as date/time variables uses the ISO 8601 standard. ISO 8601 standard is not controlled terminology, but an international formatting standard. So, text variables containing ISO 8601 date/time or duration values are not expected to have a codelist associated with them.

Table 2.6.3.2: Situations where codelists are expected

Situation	Rationale
CRF which collects whether the heart rate was measured when subject was sitting, standing or in a supine position	Variable VSPOS containing vital signs position should have the CDISC POSITION codelist associated with it according to the SDTM-IG. The codelist is created for the study with the following terms: SITTING, STANDING, and SUPINE
ACEND questionnaire which is composed of several different categories to assess physical impact: feeding/grooming/dressing, sitting/play, transfers, mobility, time, emotion, and finance	Information regarding which category the QSTESTCDs belongs to, could be stored in the grouping qualifier variable QSSCAT. As the number of assessment categories is finite, a sponsor-specific codelist should be created for QSSCAT, with the following terms: FEEDING/GROOMING/DRESSING, SITTING/PLAY, TRANSFERS, MOBILITY, TIME, EMOTION and FINANCE
Age group categories derived in the ADSL dataset	Standard ADaM variables AGEGRx and AGEGRxN contain derived age groups. As these variables are connected and the number of age groups is limited, a codelist should be created for both of them.

Situation	Rationale
Variables which stores AE coding results coded using the MedDRA dictionary	Third party dictionaries such as MedDRA have regulated terms (i.e. the same coding result would apply for the same AE under the same dictionary version regardless of sponsor or study). In a Define-XML document, this needs to be listed under the ExternalCodeList element with Dictionary name and Version specified. See Section 2.6.11 External Codelists.
Categorization of numeric measurements	Categorical or qualitative measurements where results could only be e.g. POSITIVE or NEGATIVE are expected to have codelists provided on either variable-level or value-level as it is a finite set.

Codelist Example 1: RACE codelist example to illustrate planned versus actual study data

The first column in table below displays all possible values from CDISC Controlled Terminology for RACE. Assume a study that only plans to collect what is presented in the second column. Although 'BLACK OR AFRICAN AMERICAN' race is planned to be collected, no subject with this race were enrolled in this study. As specified in the Define-XML 2.0 specification, the complete set of values relevant to the study must be provided whether they are referenced within the study data or not. Therefore, the metadata for RACE in this study should include all possible values that were planned to be collected as shown in the fourth column.

RACE CDISC Controlled Terminology	RACE values planned in the study	Actual RACE values of subjects enrolled in the study	Recommended metadata in Define-XML
AMERICAN INDIAN OR ALASKA NATIVE			
ASIAN	ASIAN	ASIAN	ASIAN
BLACK OR AFRICAN AMERICAN	BLACK OR AFRICAN AMERICAN		BLACK OR AFRICAN AMERICAN
NATIVE HAWAIIAN OR OTHER PACIFIC ISLANDER			
WHITE	WHITE	WHITE	WHITE

2.6.4. Variables without a pre-defined list of terms requiring a controlled terminology

If a variable does not have a finite set of values, e.g. it is collected as a free-text, in majority of situations it should not have a codelist associated with it. One situation to consider is when such variable should have Controlled Terminology applied to it according to an implementation guide. This usually happens when in a CRF design apart from a predefined set of items, an "Other, please specify" free-text field is available for site personnel to fill in, if they cannot find an option in the predefined set.

Codelist Guideline 1: Using Controlled Terminology in Codelist

In situations where a variable without a predefined list of values requires a standard codelist to be applied to it, every effort must be made when mapping values towards Controlled Terminologies. And in case the standard codelist is extensible, only after exhausting all of those possibilities should the value be added as a new term to the codelist. All of the new terms should be described in the Define-XML document or the Reviewer's Guide if additional clarification regarding these new terms needs to be made.

Rationale: One of the most important purposes of having Controlled Terminologies applied to variables is to standardise data. FDA instructs [10](#) that when a new concept for a codelist term is identified, sponsors should submit it to the appropriate terminology maintenance organization as soon as possible. Creation of custom terms is discouraged for a submission and if custom terms cannot be avoided, the submitter should clearly identify and define them within the submission's Define-XML document, reference them in the Reviewer's Guide, and use them consistently throughout the submission. If free-text values were not mapped to controlled terminologies where applicable but were extended freely, purpose and benefits of data standardization for the reviewers is lost.

Codelist Example 2: Units collected for local laboratories tests

A CRF for local hematology test collection was designed to have a pre-defined set of units as follows:

- 10(9)/L: which maps to the code '10^9/L' in the UNIT codelist
- 10(12)/L: which maps to the code '10^12/L' in the UNIT codelist

With an additional free-text field to collect units that are not listed above.

After extracting the production data, an SDTM mapper found out that apart from the two predefined units per CRF design, the following units were also entered by the site(s):

- THOU/uL
- 10^6/uL
- 10^3/mm3
- 10(3)/uL

Per the SDTMIG, LBORRESU needs to have a controlled terminology applied. In order to standardize data, the correct way to create the associated UNIT codelist for LBORRESU is to have all these free-text synonyms mapped accordingly:

- THOU/uL: 10^9/L
- 10^6/uL: 10^12/L
- 10^3/mm3: 10^9/L
- 10(3)/uL: 10^9/L

One would realize that after mapping has been performed, the UNIT codelist for LBORRESU would still be left with the two original predefined unit values:

- 10^9/L
- 10^12/L

However, if other unit values resulted after appropriate mapping, the additional values not in the initial set of pre-defined values would need to be added to the codelist, except for unit values entered in error that do not correspond to CDISC CT or requested to be added as extensions to CDISC CT.

2.6.5. Enumerated and Decoded Codelists

There are two types of codelists which can be specified in the Define-XML document: a codelist containing an enumeration of allowed values and a codelist containing a list of allowed values with their decodes. Both of the codelist types have identical structure and attributes apart from the additional decode value for the second type. Those codelist will be called enumeration codelist and decoded codelist.

Codelist Guideline 2: Providing Decoded Codelist

In case the coded term is an abbreviation, a code, or a score (as for questionnaires) or simply requires additional clarification, the decode codelist should be created with a decode provided for each of the terms. The NCI Preferred Term should be used as a decode value where applicable. In other cases when the coded terms are self-explanatory, an enumeration codelist should be created without decodes.

Rationale: A Decoded Codelist should only be provided if the decoded values add further clarification to the submitted code values. Related codelists discussed in Section 2.6.6 Related Codelists should also have decodes.

Note that the decode should be populated for all or none of the values within a codelist.

Codelist Example 3: Completion/Reason for Non-Completion Codelist

Codes within this codelist, such as 'LOST TO FOLLOW-UP' or 'SCREEN FAILURE' explain themselves and would not benefit the reviewer with any further clarification. Therefore, this codelist should be listed as Enumerated Codelist without decodes.

Codelist Example 4: Unit Codelist as Decoded Codelist

The following table illustrates the importance of providing unit codelist as a Decoded Codelist.

CodedValue	Decode	Comments
mg	Milligram	The abbreviation 'mg' used for CodedValue could be for Magnesium or the unit milligram. Adding a decode in this case helps reviewers
FEU	Fibrinogen Equivalent Units	FEU could be a less common unit for some reviewers therefore there are benefits of adding a decode here
%	Percentage	Although % itself is a universal known symbol however as decodes were added for other coded values within the same codelist, therefore decode is also necessary here. NCI Preferred Term is used here as a decode

2.6.6. Related Codelists

In SDTM, it is common that two or more variables are linked with each other by having one-to-one relationship between their values, for example VSTESTCD and VSTEST, ARMN and ARM, TSPARMCD and TSPARM. For the standard CDISC/CT codelist, the submission values for both code and decode for the same term on related codelists can be identified by the same C-code.

For example, in the case of Trial Summary Parameter Test Code (TSPARMCD) and Trial Summary Parameter Test Name (TSPARM), NCI Code for "ACTSUB" submission value in TSPARMCD codelist and "Actual Number of Subjects" submission value in TSPARM codelist share the same C-code ("C98703"). Note that each of the related codelists have different C-codes: TSPARMCD's C-code is "C66738" and TSPARM's C-code is "C67152" (Refer to the 2019-03-29 CDISC CT).

Codelist Guideline 3

When there are two variables containing a coded value and a related decoded value, codelists should be created for both variables.

Rationale: In most cases, coded terms within codelists such as VSTEST and TSPARM are self-explanatory and are just an enumeration of allowed values which do not require additional decodes. Such coded terms could simply be represented using enumeration codelists. On the other hand, codelists such as VSTESTCD and TSPARMCD contain abbreviated terms and it is helpful for reviewers to provide decodes for these terms (i.e. the VSTEST and TSPARM). Such coded terms need to be represented using decoded codelists. Ability to add decodes is not limited only for the related codelists. Authors of Define-XML documents should consider how reviewers would benefit from decodes of coded terms and add them appropriately.

Codelist Example 5: Parameter Categories

A study, as part of its neurological examination, collects data in the areas of mental status, level of consciousness, sensory function, motor function, cranial nerve function and reflexes. The analysis tables requested for these assessments to be presented in a certain order therefore apart from creating PARCAT1 in the analysis dataset, PARCAT1N was created for ordering.

Below table illustrates when enumeration and decodes should be used:

Variable Name	CodelistID
mg	PARCAT1_NEURO This codelist was created which simply shows an enumeration of allowed values: General Cranial Nerves Reflexes Plantar Response Motor System Coordination/Cerebellar Function Sensation-Temperature Sensation-Vibration
PARCAT1N	PARCAT1N_NEURO This codelist was created with decodes provided showing what each numeric value meant: 1=General 2=Cranial Nerves 3=Reflexes 4=Plantar Response 5=Motor System 6=Coordination/Cerebellar Function 7=Sensation-Temperature 8=Sensation-Vibration

2.6.7. Variable-level and Value-level Codelists

The value-level metadata allows to describe variable metadata specific in a particular situation, for example it allows to describe metadata for a character result variable depending on a test. In this case there are 3 possible ways to provide a codelist:

- Codelist is specified on the value-level and contain terms specific only for it
- Codelist is specified on a variable-level and contains a superset of terms for all value-levels
- The codelist is specified for both variable and value-levels as described above.

Codelist Guideline 4: Codelist at Value-level

When a variable has value-level specified and value-level items have different codelists associated with them, the codelists should be provided on the value-level definition.

Rationale: Specification of a codelists on the value-level allows better understanding and review of the metadata.

Codelist Example 6: CGI Questionnaire

A study collects CGI in the following areas in order to assess subject's global functioning after initiation of the study treatment:

Codelist Example 6: CGI Questionnaire

A study collects CGI in the following areas in order to assess subject's global functioning after initiation of the study treatment:

Question	Allowed Responses
Severity of Illness	0=Not assessed 1=Normal, not at all ill 2=Borderline mentally ill 3=Mildly ill 4=Moderately ill 5=Markedly ill 6=Severely ill 7=Among the most extremely ill patients
Global Improvement	0=Not assessed 1=Very much improved 2=Much improved 3=Minimally improved 4=No change 5=Minimally worse 6=Much worse 7=Very much worse

Two different codelists should be created on the value-level instead of a single codelist on the variable-level for two reasons:

1. Reviewers will be able to understand the fact that responses are different depending on the question.
2. Some decodes of the same codes are different. For example, '1' could be 'Normal' or 'Very much improved' depending on the question.

As a result, two codelists needs to be created on the value-level as follows:

Variable Name	Value-Level Where Clause	Label	CodeListID
QSTESTCD		Question Short Name	QSTESTCD Including: CGI0101=Severity of Illness CGI0102=Global Improvement
QSORRES	QSTESTCD EQ CGI0101 (Severity of Illness)	Finding in Original Units	CGI0101N Including: 0=Not assessed 1=Normal, not at all ill 2=Borderline mentally ill 3=Mildly ill 4=Moderately ill 5=Markedly ill 6=Severely ill 7=Among the most extremely ill patients
QSORRES	QSTESTCD EQ CGI0102 (Global Improvement)	Finding in Original Units	CGI0102N Including: 0=Not assessed 1=Very much improved 2=Much improved 3=Minimally improved 4=No change 5=Minimally worse 6=Much worse 7=Very much worse

There can be situations where some value-levels need to have a codelist applied to them, while other don't. In this case, even if several value-levels share the same codelists, this codelist must be specified on the value-level and not on the variable-level. Specifying it on the variable-level implies that the codelist is applicable for all value-levels.

Codelist Example 7: Analysis Adverse Event

There are 3 parameters derived for each subject: whether the subject experienced any adverse events (Y/N), whether the subject experienced any serious adverse events (Y/N), and the number of adverse events. The last parameter does not have a codelist associated with it and the other two parameters have the same codelist. Still the codelist cannot be specified on the variable-level, because if specified it would mean that that codelist applies to all 3 parameters, including the number of adverse event parameters.

Variable Name	Where Condition	CodeListID
AVALC		(No codelist is applied)
AVALC	PARAMCD IN ("AEFL" (Subject Experienced Any AE Flag), "SAEFL" (Subject Experienced Any SAE Flag))	NY_AVALC Note: No Yes Response, Study Subset for AVALC: Y = Yes, N = No See Section 2.6.10.1
AVALC	PARAMCD EQ AENUM (Number of AE)	(No codelist is applied)

Codelist Guideline 5: Codelist at Variable-level

When a variable has value-level metadata specified and all value-level definitions have the same codelist associated with them, the codelist should be provided on the variable-level.

Rationale: It would be clearer that there is no difference in the definition regarding the values allowed for each value-level definition.

Codelist Example 8: WHO Motor Milestones

In order to assess a subject's motor milestones achieved, a study uses the following WHO criteria: Sitting without support, Hands and Knees Crawling, Standing with Assistance, Walking with Assistance, Standing Alone, and Walking Alone.

For each of these tasks, examiner needs to observe and provide a response, where:

- 1 = No (inability)
- 2 = No (refusal)
- 3 = Yes
- 9 = Unable to test

All possible responses for all of the WHO criteria are the same, therefore providing a codelist on the value-level does not bring any additional value for the reviewers (or even, creating a VLM in this case does not bring any additional value). Therefore, it is sufficient to create a codelist which describes the above response values at the variable-level.

Note that when a codelist is provided on the variable-level, all value-levels must have a finite set of variable values.

2.6.8. Codelist Versioning

All codelists should be versioned based on the CDISC CT version release dates. This should enable trial teams to select the CDISC CT version used for their trial, which is to be reported to regulatory authorities in the Data Reviewer's Guide when submitting data. It is recommended to include the CDISC CT version date in the Metadata Description. Refer to Section 2.1.2.1 Study-level Metadata.

Sponsors should use the most recent version of the CT at the start of the study [11](#). After that, it is a sponsor decision whether to update the CT version or stay with the original version.

- CDISC CT updates are released each quarter. However, the Define-XML document should reflect the version used in the study.
- It is expected that all terminology used in a study comes from the same CDISC CT version date within one model but may be different for different models. For example, when creating the Define-XML document for ADaM, the version of the SDTM CT used in the file can be different from the ADaM CT version.

With each new CDISC CT version, a full assessment is required regarding the impact on the existing codelists: additions, replacements, updates and deletions. Not only codelists from CDISC CT should be reviewed during the update. It also needs to be checked if sponsor-defined terminology should be revisited based on new codelists and coded values released.

When the data is pooled, a single version of CT should be used for each of the models. The example below shows two studies pooled together for the purpose of an integrated summary of safety:

Study	SDTM CT Version	ADaM CT Version
Study A	2016-12-16	2017-03-31
Study B	2017-03-31	2017-09-29
Study A and B ISS	2017-03-31	2017-09-29

2.6.9. Sponsor-defined CT / Extension of CDISC CT

Although CDISC CT includes a large number of codelists and new codelists and updates to the existing codelists are frequently released, a situation that often occurs in studies is that there is no suitable codelist released by CDISC for a particular item. For example, a study-specific questionnaire that is not included in the QRS CT12. In such situations, a sponsor-defined codelist should be created.

Codelist Example 9: WHO Motor Milestones

Continuing with the WHO Motor Milestones example from the previous section, there are no suitable CDISC codelists that could be applied for the responses. Therefore, a sponsor-defined codelist is created as follows on a variable-level, assuming the meaning of the numbers 1, 2, 3, 9 represent categories instead of numeric scores:

Variable Name	Label	CodeListID
QSORRES	Finding in Original Units	WMMORES: Codelist WMMORES with decodes is created: 1 = No (Inability) 2 = No (Refusal) 3 = Yes Note that the response: 9=Unable to test was mapped to QSSTAT=NOT DONE with QSREASND=UNABLE TO TEST and should not be mentioned in the codelist.
QSSTRESC	Character Result/ Finding in Std Format	WMMORES Note: The same codelist WMMORES is used as for QSORRES
QSSTRESN	Numeric Finding in Standard Units	Note: Since the options do not represent numeric scores, it is not correct to populate QSSTRESN. Refer to the SDTMIG v3.2, CDISC Notes for QSSTRESN. This case is different from the HAMD-17 QS where the meaning of 1 to 4 are numeric scores.

Another situation that might occur is when the majority of the terms can be mapped to a CDISC codelist, but there are some terms that are not part of the codelist. In this case, the CDISC codelist can be extended with sponsor-defined terms, given that this codelist is marked as extensible in the CDISC CT. Terms that are added to the existing codelists need to be indicated in the Define-XML document (using the extended value attribute of a code) and described in the Data Reviewer's Guide.

Codelist Example 10: Needle Insertion Sites

As part of cerebrospinal fluid sampling, a location where a lumbar procedure is performed is collected in a study. When mapping this location into the PRLOC variable, the SDTM mapper finds out that there is one location that is not present in CDISC Codelist LOC (Anatomical Location) which should be used for the PRLOC variable according to the SDTMIG:

CRF Option	Corresponding CDISC Term in 'LOC'	CDISC Codelist 'LOC' Extended?
L2/L3	L2-L3 INTERVERTEBRAL SPACE	
L3/L4	L3-L4 INTERVERTEBRAL SPACE	
L4/L5	L4-L5 INTERVERTEBRAL SPACE	
L5/S1		Yes, the term 'L5 INTERVERTEBRAL SPACE - S1 SACRUM' is added to the CDISC LOC codelist as this codelist is extensible

Although extending CDISC codelists is mainly applicable only for those CDISC codelists that are marked as extensible, the SDTMIG 13 recommends that, in case multiple responses are allowed for a variable, the value "MULTIPLE" is to be added to the non-extensible codelist e.g. CodeList="CAN". When adding sponsor-defined values to a CDISC codelist, ensure there is no overlap with CDISC values.

2.6.10. Codelist Subsets

Codelists defined in the CDISC Controlled Terminology may be supersets of terms used across different variables, datasets, and studies. Applying a CDISC Controlled Terminology codelist, or sponsor defined terminology, to a variable in a dataset for a particular study, could imply the definition of subsets that can be used on only one variable or multiple variables in the dataset or across datasets for the given study. This section is intended to describe different cases of subsets definitions other than already published by CDISC CT.

The CDISC Implementation Guides indicate a list of pre-specified codelists which should be used for variables when they are presented in the study metadata. It is common that the same codelist is used across multiple variables. As discussed in the previous sections, the study codelists must contain only those terms which are allowed for that study. However, the degree of granularity does not stop at the study level and can be increased to dataset, variable-level and value-level definitions. An example is the UNIT codelist which has several hundred terms listed in the CDISC Controlled Terminology. Only units which are relevant to a study should be present in codelist definitions of the study metadata. But different domains use different sets of units and it is important to distinguish those sets in the metadata to allow a reviewer to understand what the permissible units in each particular situation are. Units referenced in the electrocardiogram domain are different from those used in the concomitant medications or laboratory test results domains. To address this, different subsets of the UNIT codelist should be

created, where each subset codelist defines a unique list of units used for a particular result.

Creation of the codelist subsets is not restricted to CDISC Controlled Terminology. If there is a sponsor-defined codelist which has different sets of terms used across different variables on a study, then different subsets of the sponsor-defined codelist should be created as well.

Codelist Guideline 6: Varying Codelist Subsets

When applying the same codelist to more than one variable in a study and those variables have different sets of permissible values, different subsets of the codelist should be created.

Rationale: Reviewers need to understand what the set of allowed values in each particular situation is. Validation systems use codelists to verify that there are no unexpected values in the data.

Codelist Example 11: Unit Codelists

The UNIT codelist per the SDTM Implementation Guide is used across several unit variables such as --ORRESU and --STRESU. It is most likely that units used in the CM domain are different from those used in the LB domain. It is recommended to create a unique list of units (UNIT_CM) specific to the CM domain to distinguish it from the LB units. Also, in the LB domain (and many other domains where standard results are stored), the list of units used for LBORRESU may be different from those used in LBSTRESU. In those cases, it is recommended to create one subset for LBORRESU (UNIT_LBORRESU) and another subset for LBSTRESU (UNIT_LBSTRESU).

CodeListID	CodeList Name	CodedValues	Notes
UNIT_LBORRESU	Unit, Study Subset for LB.LBORRESU	%, mg, mL	This subset could only be used for LB.LBORRESU
UNIT_LBSTRESU	Unit, Study Subset for LBSTRESU	%, L, g	This subset could only be used for LB.LBSTRESU
UNIT_CM	Unit, Study Subset for Concomitant Medications	%, PELLET, L, TABLET, g, mg, mL	Although majority of units used are within UNIT_LBORRESU and UNIT_LBSTRESU, but as there are additional units such as PELLET and TABLET therefore this is considered to be a different subset. In this case creation of a new codelist subset is helpful for reviewers to differentiate between units used in the CM and LB domains

Codelist Example 12: Occurrence (--OCCUR) and subject-level population flag (--FL) variables

Allowed values for Occurrence (--OCCUR) variables are Y/N/null and Y/N for ADaM population flag (--FL) variables. Given that null values are never used as part of a codelist, the same codelist subset can be for both types of these variables. As the list of allowed values is different from the one given in Example 1, another subset of the NY codelist, 'NY_NY' should be created and used across all applicable variables:

CodeListID	CodeList Name	CodedValues	Example of Variables Applicable
NY_NY	No Yes Response, Subset for Yes and No	N, Y	CMOCCUR, ECOCCUR, MHOCCUR, ADaM --FL variables which allows only Y/N/null or Y/N values, etc.

There are situations, when creation of different subsets does not bring any additional value for reviewers. Example is the DOMAIN variable and associated DOMAIN codelist subsets, which contain only one term for each domain. In these situations, it is possible to specify a codelist, containing a superset of all values used within the study metadata.

Guideline: When variables have the same name across different domains and are representing the same type of study data, a superset should be created, which will be applied to all these variables.

Rationale: Multiple subsets specific to each domain and created for the same variable does not make understanding of the define easier, given that it is possible to understand which subset is required in each particular case.

Codelist Example 13: Epoch codelist

In case variable names are the same across different domains and they represent the same type of data, it is allowed to unite them into a superset codelist.

CodeListID	CodeList Name	CodedValues	Notes
EPOCH	Epoch	SCREENING, TREATMENT, FOLLOW-UP	This codelist contain a superset of all epochs on a study. Therefore, it can be applied to all EPOCH variable in the study.

Note that this guideline should not be applied only based on a variable name. For example, different analysis value (AVALC) variables in different datasets in most situations will represent different type of data, therefore different subsets should be created where applicable. The following two codelist subsets are assigned to variables with the same name (ADPAIN.AVALC and ADLB.AVALC) but are not united into a superset as they represent different types of data.

CodeListID	CodeList Name	CodedValues	Notes
PAINSEV	Pain Severity	Mild, Moderate, Severe	A codelist used to represent for pain severity. Applied for ADPAIN.AVALC
DISSTAT	Disease Presence Status	Positive, Negative	A codelist used to represent test result for disease presence. Applied for ADLB.AVALC

2.6.10.1. Naming Codelist Subsets

Codelist Guideline 7: Codelist Subsets Naming Convention

When there is more than one subset of the same codelist used in the study metadata, it is recommended to specify the CodeList Name as the CodeList Name of the parent codelist and add a description for which purpose the subset is created using the following format: <CodeList Name of the parent codelist>, <purpose of the subset>.

Rationale: It should be possible to identify subsets, which are based on the same parent codelist and understand reason for the subset creation.

For example, when a codelist subset is applied to a specific variable:

- “Unit, Study Subset Used for LB.LBSTRESU”.

When the same codelist subset is used for more than one variable, a meaningful description should be used instead of a variable name:

- “Date Imputation Flag, Study Subset for Day and Month”
- “Specimen Type, Study Subset for PK and Immunogenicity”

Codelists Guideline 8: Single Codelist Subset

If there is only one subset of a codelist used in the study metadata, then it is recommended not to mention that it is a subset in the codelist name.

Rationale: There is no requirement to identify that a codelist used in the study metadata is a subset or a full copy of the parent codelist. Naming it the same way as the standard full codelist makes it easier to maintain codelists within a study metadata.

For example, if the entire study only collects specimen condition in LB domain’s LBSPCCND, then there is no need to create a subset of the SPECCOND codelist just for the LB domain as this codelist would not be used elsewhere.

When creating codelists, it should be verified that there are no codelists with different names but represent the same list of terms and kind of data. Still there may be situations, when two

different codelists, representing different types of data, can have the same list of terms.

Codelist Example 14: Usage of Codelist with the same list of terms

If different variables have the same list of allowed values and those values have the same meaning, then they should be united into one codelist. As an example, the Baseline flag and pre-specified variables only allows the value of Y/null, for which a subset of the NY CodeList, ‘NY_Y’ should be created and used across all --BLFL and --PRES variables instead of creating two codelist subsets with the same list of terms.

CodeListID	CodeList Name	CodedValues	Notes
NY_BL	No Yes Response, Study Subset for Baseline	Y	These codelists contain the same list of terms and represent the same type of data, therefore they should be united and used as one codelist
NY_PRESP	No Yes Response, Study Subset for Prespecified	Y	

A single codelist should be created:

CodeListID	CodeList Name	CodedValues	Example of Variables Applicable
NY_Y	No Yes Response, Study Subset for Yes only	Y	PEBLFL, LBBFL, PRPRES, CMPRESP, ADaM --FL variables which allows only Y/null values, etc.

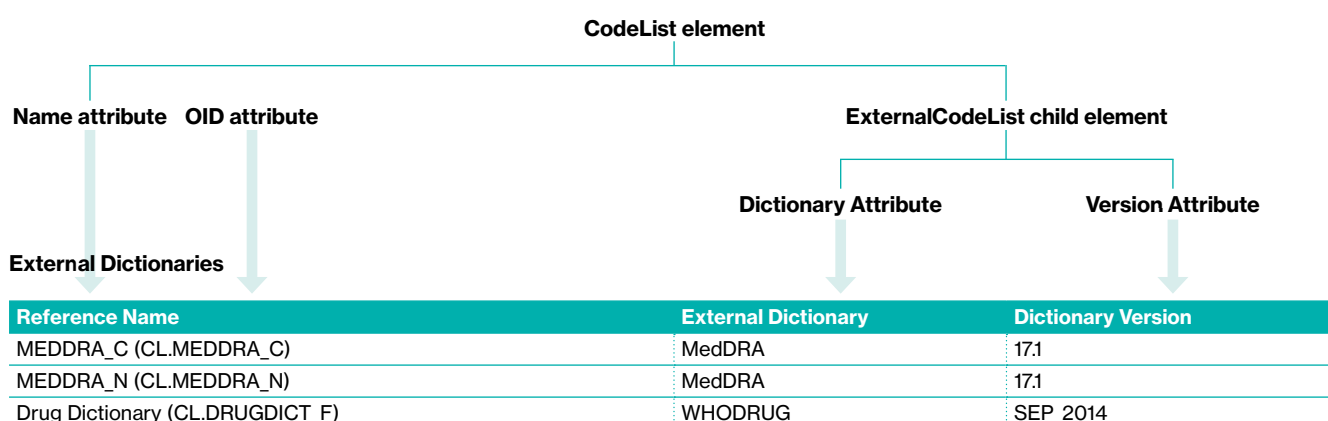
There are situations when different variables have the same list of allowed values, but those values have different meaning, as currently it may happen with some CDISC/CT terms. Note these cases are identified with same terms having different C-codes and corresponding CDISC Definition. In this case they should remain as independent codelists.

CodeListID	CodeList Name	CodedValues	Notes
NRIND	Reference Range Indicator, Study Subset for LB.LBNRIND	NORMAL, ABNORMAL	These codelists contain the same list of terms, still as they represent different types of data, they should not be united.
NORMABNM	Normal Abnormal Response, Study Subset for Response	NORMAL, ABNORMAL	

2.6.11. External Codelists

Codelists, such as coding dictionaries, provided by third party vendors are referred as 'External Codelist' in the Define-XML document. They require a different type of the information provided in the document comparing to other CDISC or sponsor-defined Controlled Terminologies.

2.6.11.1. Elements of the External Codelists Metadata



There are two mandatory attributes required for each of the external codelists: name and version of the dictionary used. If a public URL of the dictionary used is available, then it should be specified using optional attribute 'href'. If the dictionary is included in the submission a path to it (relative to the Define-XML document) should be provided using attribute 'ref'.

There are also some considerations that needs to be taken when assigning external codelists:

Coding dictionaries, such as MedDRA, usually have results that comes in pairs, one to represent the text description and the other to represent the code value associated with it. If the data types of variables that use the dictionary are different, then it is required that separate codelists, which references the same dictionary, are created for each of the data types.

Codelist Example 15: MedDRA coding for AE

Below shows how two different external codelists are assigned for lower level terms variables in AE, coded using the MedDRA dictionary version 20:

Variable Name	Label	Data Type	External Codelist Assigned
AELLT	Lowest Level Term	text	MEDDRA_C
AELLTCD	Lowest Level Term Code	integer	MEDDRA_N

The metadata displayed with a stylesheet could be presented as:

External Dictionaries

Reference Name	External Dictionary	Dictionary Version
MEDDRA_C (CL.MEDRA_C)	MedDRA	20.0
MEDDRA_N (CL.MEDRA_N)	MedDRA	20.0

Unlike ISO 8601 which is a representation standard for date, time, and duration formats, ISO 3166 standard for country codes contains a finite set of internationally recognized codes of letters and/or numbers ¹⁴. Variables, such as SDTM variable DM.COUNTRY, use that standard to represent list of countries in a three-letter (alpha-3) code format. Since release of the 2016-09-30 controlled terminology, the CDISC "COUNTRY" codelist was removed as the country identifiers should be standardized by ISO 3166. In order to adapt to this change, ISO 3166 should now be specified as an external codelist.

Codelist Example 16: Country Codes for DM domain COUNTRY variable

For example, in the DM domain COUNTRY variable associate the ItemDef element with reference to the external codelist created for ISO 3166.

External Reference (Name)	External Reference (Data Type)	Dictionary Name	Dictionary Version	External Reference (href)
Country Codes (ISO 3166)	text	ISO 3166	2013-11-15	http://www.iso.org/iso-3166-country-codes.html

2.7. Methods**2.7.1. Methods Metadata**

The Methods metadata is intended to describe unique computation algorithms used to populate dataset variables. Methods must be provided for variable- and value-level definitions with Derived origins.

For additional method definition examples, refer to Section 4.6.1 Examples of Computational Method Definition in the Define-XML 2.0 Specification.

The following items are included as clarification notes:

- Methods can be referenced by multiple variable- or value-level items. In these cases, it is best to avoid selecting specific variables in the textual description or formal expression, unless they are common to all the referencing metadata items. Instead, reference source variables in generic terms (for example by replacing the domain prefix with '--' as in '--DY').
- Methods should only reference variables used in the submission.

2.7.2. Components of Methods Metadata

Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
Method ID	Unique ID to identify each unique method. The Define-XML Specification does not provide an attribute to include this Metadata Item. Note Method ID is not the same as the OID that is reference in the Define-XML Specification. See Section 1.3 How to Use This Document.	"AETRTEM", "CLSIG", "abc123"	
Method Name	The name of the method. It can be used as a label to summarize the use of the method.	"Algorithm to derive the AETRTEM flag" "Algorithm to assess clinical significance", if only one algorithm is provided to assess clinical significance for all results.	5.3.13. MethodDef Element section, Name attribute.
Method Type	Per SDTMIG, SDTM based data must not be imputed.	"Computation", "Imputation"	5.3.13. MethodDef Element section, Type attribute.
Method Description	The textual description of a unique method. This field is required.	"AETRTEM = 'Y' if Adverse Event was not present prior to the RFSTDTC, or it was present prior to the RFSTDTC but increased in severity during the treatment period. Null otherwise." "Only created if value qualifies as potentially clinically significant high or low based on the high and low ranges specified in the Statistical Analysis Plan."	5.3.13. MethodDef Element section, child Description element, 5.3.10.1. Description Element
External Document Reference	A reference to an external document, included in the submission data package, that contains additional details or more complete documentation of the method. Refer to Section 2.2.1 Specifying Supporting Documents-level Metadata for more information.		5.3.6.1. def:DocumentRef Element
FormalExpression	Formal expressions are optional in the Define-XML 2.0 Specification; however, it is a helpful metadata item to provide the actual software code to populate a given variable or value-level item.	Refer to the Define-XML 2.0 Specification, 4.6.1.4. Example of Method Definition with FormalExpression included.	5.3.13. MethodDef Element section, child 5.3.13.1. FormalExpression element.
Formal Expression Context	See Section 2.6.11 External Codelists.	Refer to the Define-XML 2.0 Specification, 4.6.1.4. Example of Method Definition with FormalExpression included.	5.3.13. MethodDef Element section, child 5.3.13.1. FormalExpression element, Context attribute.

2.8. Comments

2.8.1. How Comments Work

Comment text under the Define-XML 2.0 standard can be attached to most metadata items. Comments are referred to by an identifier, which means that a specific comment can be attached to more than one metadata item.

Comments can be attached to datasets, variable-level or value-level definitions and Where Conditions.

Comments are always optional except for where conditions across datasets. They provide information that does not easily

fit into other metadata items. They can describe the reason a dataset or variable exists, what part the dataset or variable plays in analysis, its relationship to other datasets or variables, or any other important contextual information.

Comments can contain references to other documents such as reviewer's guides or statistical analysis plans. In rare occasions, comments could also reference aCRF documents.

2.8.2. Components of Comments Metadata

Comment Metadata Item	Notes	Examples	Define-XML 2.0 Specification Reference
Comment	A plain-text comment. This contains the text of the comment. If long explanations are needed, the comment can contain a short descriptive statement and the DocumentRef can contain a reference to an external document such as a reviewer's guide. In this case, the text can also explain where the link will lead to when clicked. Software-specific code should be avoided here. If software-specific code is needed, it should be documented in MethodExpression. Users are encouraged to not repeat CommentText using different Comment IDs; instead, using the same reference where applicable.	"Numeric code for TRT01P which corresponds to the randomized dose"	5.3.10.1 Description Element, 5.3.10.1.1 TranslatedText Element
DocumentRef	Reference to an external PDF document (optional). See Section 2.2.1 Specifying Supporting Documents-level Metadata for details about how to set up external document references. If one is included, the referenced PDF document should contain comment text.		5.3.6.1 def:DocumentRef Element

2.8.4. Examples of Comments Metadata

At the dataset level, comments can provide context for the dataset as a whole. For example, a domain comment could be used to describe the purpose an analysis dataset serves in relation to the SAP.

Dataset Name	Description	Comment
ADTTE	Time to Event Analysis Dataset	Used for analysis of overall survival and progression free survival. See SAP section x.x

(Note: The text in blue underline is meant to be hyperlinked to the relevant section of the document.)

A dataset comment can also provide an explanation of what is contained in a dataset. For example, it is often necessary to split a source dataset into more than one file for submission. A comment can explain how the records were separated and what the resulting datasets contain.

Dataset Name	Description	Comment
QSCS	Cornell Scale Questionnaire Dataset	QS is submitted as a split dataset. The split was done based on QSCAT. QSCS contains records where QSCAT= "CORNELL SCALE FOR DEPRESSION IN DEMENTIA".
QSMM	MMSE Questionnaire Dataset	QS is submitted as a split dataset. The split was done based on QSCAT. QSMM contains records where QSCAT= "MINI MENTAL STATE EXAMINATION".

At the variable-level or value-level, a comment can be used to provide information that doesn't belong in the method. Since methods generally describe how a variable is defined, comments may be needed to explain why this variable or value exists, what it is used for, how it relates to other variables or values, or any other important notes that aren't part of the derivation description.

For example, a variable-level entry for a population flag variable in ADSL may need to describe both how values are derived and also what they are used for.

Variable Name	Method	Comment
ADSL.SAFFL	Set to 'Y' if EX.EXDOSE contains at least one administered dose, otherwise set to 'N'	Use to identify the subject population for all safety analyses.

In this case, the method describes the algorithm used to populate values in the variable, but the comment explains what part the variable plays in the analysis.

Comments describing the usage of a variable can be particularly handy in cases where the standard label of a variable does not make it clear what the variable is intended for. For example, analyzed-record flags in ADaM are required to have specific variable labels, which are not always informative. Comments can provide this detail. In the example below, ADaM rules prevent us from assigning a descriptive variable label. The comment allows us to explain the significance of a 'Y' value.

Variable Name	Method	Method	Comment
ADLB.ANL01FL	Analyzed Record Flag 01	Set to 'Y' if AVISITN is between 2 and 10	Selects lab values in the active treatment period.

Appendix 1: Value-level Metadata Display Example Using Default Stylesheet

VS (Vital Signs) – FINDINGS

Location: vs.xpt

Related Supplemental Qualifiers Dataset: SUPPVS (Supplemental Qualifiers for VS)						
Variable	Where Condition	Label/Description	Type	Length [Significant Digits] : Display Format	Controlled Terms or ISO Format	Origin/Source/Method/Comment
STUDYID		Study Identifier	text	7		[Origin] Protocol
VSORRES VLM		Result or Finding in Original Units	text	30		[Origin] CRF Annotated Case Report Form [11]
	VSTESTCD = "DIABP" (Diastolic Blood Pressure)	Diastolic Blood Pressure (Orig U)	integer	2		[Origin] CRF Annotated Case Report Form [11]
	VSTESTCD = "HEIGHT" (Height)	Height (Orig U)	float	5 [1] : 5.1		[Origin] CRF Annotated Case Report Form [11]

Appendix 2: Completion Guidelines Development Team

Name	Company or Sector	Other CDISC / PHUSE Participation ¹⁵
Core Team		
Marcelina Hungria (Co-lead)	DIcore Group	CDISC Define-XML, CDISC MSG Team, CDISC SDS and PHUSE Define-XML 2.0 Stylesheet Recommendations teams
Prafulla Girase (Co-lead)	Biogen	CDISC ADaM Team
Dmitry Kolosov	PAREXEL	PHUSE Define-XML 2.0 Stylesheet Recommendations Team (Co-lead)
Sherry Chou	PAREXEL	
John Brega	PharmaStat	
Linda Collins	PharmaStat	CDISC ADaM Team
Special Contribution		
David Brega	PharmaStat	
Lex Jansen	SAS Institute	CDISC Define-XML Team, PHUSE Define-XML 2.0 Stylesheet Recommendations (Co-lead)
Veerle Coenen	Janssen	
Majdoub Haloui	Merck	
Jennifer Feldman	Epreda	CDISC SEND Team
Satheesh Avvaru	MaxistIT	
Lisa Brooks, Jane Lozano	Iris Statistical Computing, Eli Lilly and Company	PHUSE ODS Team (Co-leads)
Other participants over time ¹⁶	Industry, Academic, FDA	
Members from other teams for content discussions / clarifications		CDISC Define-XML, CDISC MSG and CDISC SDS teams
Reviewers during Public Review period ¹⁷	XMLPharma, GSK, HMS Analytical Software GmbH, PRA Health Sciences	CDISC Define-XML Team, PHUSE Define-XML 2.0 Stylesheet Recommendations Team

References

- The Define-XML V2.0 Completion Guidelines project was initiated along with the PHUSE Define-XML V2.0 Style Sheet Recommendation project, as part of the Optimizing the Use of Data Standards (ODS) Working Group. See Section 1.4 Glossary and References.
- CDISC ODM version 1.3.2, section 2.8
- The official Define-XML Specification has a different definition for the StudyName and ProtocolName attributes. It was discussed with ODM team that this is the intended purpose of these attributes.
- FDA Study Data Standards Resources
- Note that, as per regulatory requirements at the time this document is being written, it is expected that this kind of information will be provided in the reviewer's guide. However, users are encouraged to include this information as part of the Metadata Description item in all Define-XML 2.0 documents.
- The draft Define-XML 2.1 Standard, which is undergoing public review period at the time this document is being written, already includes structured metadata items to provide this information in a standardized way.
- Refer to the latest regulatory guidance documents. For example, refer to Section 2.3 Analysis Data Reviewer's Guide, last sentence, in the FDA TCG v4.2.
- Per clarification from the PHUSE Define-XML team. In Section 4.4 Value Level Metadata Definitions of the Define-XML 2.0 specification, the example given for the Length attribute is invalid as Length is a required attribute for specific DataTypes. So even if the value-level Length is not different from the variable-level Length, it still must be provided on both levels if DataType is one of the following: text, integer, or float.
- This principle applies to all optional ItemDef attributes and MethodOID, Mandatory, Role, RoleCodeListOid attributes of ItemRef, as well as Label, Origin, and CodeListRef subelements of ItemDef.10. <http://www.obofoundry.org/>
- Study Data Technical Conformance Guide, Section 6.1.3
- Study Data Technical Conformance Guide, section 6.1.2
- Questionnaires, Ratings and Scales. <https://www.cdisc.org/foundational/qrs>
- SDTMIG 3.2 section 4.1.2.8
- <https://www.iso.org/iso-3166-country-codes.html>
- Participation in team relevant to the Completion Guidelines content
- See complete list on the PHUSE Wiki Project page
- See feedback on the PHUSE Wiki Project Page